



Budapest University of Technology and Economics
Faculty of Transportation and Vehicle Engineering
Department of Automotive Technologies



Master thesis

Development, Implementation and Testing of a Collision Avoidance System for Autonomous Vehicles.

Author:

Gustavo Henrique Moers

Supervisor:

Dr. Ádám Bárdos

Budapest, 2024.

Thesis Task

Name: Gustavo Henrique Moers (IBMYA1) **Mode:** Autonomous Vehicle Control Engineer MSc (Full-time)

Title: Development, Implementation, and Testing of a Collision Avoidance System for Autonomous Vehicles.

Serial number: GJT-M-G-2024-9

Task to be developed:

1. Conduct a thorough and systematic review of the existing literature pertaining to collision avoidance systems in the context of autonomous vehicles and comprehensively assess the state-of-the-art in this field.
2. Explore and provide an overview of the legal and regulatory framework pertaining to autonomous vehicles and their collision avoidance systems, analyzing international, national, and regional laws, regulations, and standards that pertain to the deployment, operation and homologation of such systems.
3. Create a simulation environment dedicated to testing and evaluating the collision avoidance system.
4. Define the requirements and operational domain of the system and based on it design and implement the algorithms of the collision avoidance system.
5. Execute testing and evaluation of the collision avoidance system within the established simulation environment, employing pertinent metrics regarding collision prevention and analyzing the obtained results to draw conclusions regarding the system's performance and limitations.

Supervisor: Dr. Ádám Bárdos

Organization, Department: BME Department of Automotive Technologies

Subjects of the final exam:

1. Automotive vehicle systems (BMEKOGGM712)
2. Vehicle Dynamics/Vehicle mechanics fundamentals (BMEKOGGM705/ BMEKOGGM713)
3. Automotive environment sensors (BMEKOKAM708)

Deadline for the thesis submission: 24 May 2024

P.H.

Dr. Szalay Zsolt
associate professor
head of department

Declaration

I, Gustavo Henrique Moers, hereby declare that the present master's thesis was composed by myself and that the work contained here in is my own. I also confirm that I have only used the specified resources. Any part that I have taken from another source, either verbatim or in the same content, but reworded, has been clearly marked with the source indicated.

.....
Gustavo Henrique Moers

Abstract

This thesis focuses on the development, implementation, and testing of a Collision Avoidance System for Autonomous Vehicles. The research consists of a literature review incorporating both classical and learning-based approaches. Additionally, the study examines the legal and regulatory frameworks impacting autonomous vehicle technologies, providing a comprehensive overview of international standards and requirements. The core of the thesis introduces a hybrid system that combines Reinforcement Learning (RL) and Model Predictive Control (MPC) within the CARLA simulation environment, alongside a comparison with an end-to-end control strategy that maps raw sensory inputs directly to vehicular control. This system is tested to evaluate its collision prevention capabilities under simulated conditions. The findings support a synergistic integration of machine learning and traditional control systems to advance the safety and operational efficiency of autonomous driving technologies.

Table of Contents

1. INTRODUCTION.....	7
2. STATE-OF-THE-ART IN COLLISION AVOIDANCE SYSTEMS FOR AUTONOMOUS VEHICLES	10
2.1. CLASSICAL APPROACHES	14
2.1.1 GRAPH SEARCH ALGORITHM.....	15
2.1.2 CURVE INTERPOLATION ALGORITHM	16
2.1.3 SAMPLING-BASED APPROACH	18
2.1.4 ARTIFICIAL POTENTIAL FIELD METHOD	19
2.1.5 NUMERICAL OPTIMIZATION ALGORITHM	20
2.2. LEARNING APPROACHES.....	21
2.2.1 SUPERVISED LEARNING	22
2.2.2 REINFORCEMENT LEARNING.....	24
2.3. COMPARATIVE ANALYSIS OF COLLISION AVOIDANCE METHODS FOR AUTONOMOUS VEHICLES.....	26
2.4. ADVANCED COLLISION AVOIDANCE STRATEGIES FOR AUTONOMOUS VEHICLES	28
3. LEGAL AND REGULATORY FRAMEWORK FOR COLLISION AVOIDANCE SYSTEMS.	31
3.1. EURO NCAP 2025 ROADMAP.....	32
3.2. UN REGULATION NO. 152 - ADVANCED EMERGENCY BRAKING SYSTEM (AEBS).....	35
3.3. UN REGULATION NO. 79 - STEERING EQUIPMENT.....	36
3.3.1 AUTOMATICALLY COMMANDED STEERING FUNCTION (ACSF).....	36
3.3.2 CORRECTIVE STEERING FUNCTION (CSF).....	37
3.3.3 EMERGENCY STEERING FUNCTION (ESF).....	37
4. RESEARCH OBJECTIVES AND METHODOLOGY	40
4.1. CARLA SIMULATION ENVIRONMENT	40
4.2. TRAINING SCENARIO	41
4.3. PROXIMAL POLICY OPTIMIZATION (PPO)	42
4.4. REWARD FUNCTION	44
5. DESIGN AND IMPLEMENTATION OF A HYBRID COLLISION AVOIDANCE SYSTEM USING REINFORCEMENT LEARNING AND MODEL PREDICTIVE CONTROL	47
5.1. PROPOSED FRAMEWORK.....	47
5.2. SCOPE AND LIMITATIONS	48
5.3. STATE SPACE	49
5.4. ACTION SPACE	51
5.5. TRAJECTORY PLANNER.....	52
5.5.1 CUBIC SPLINE CURVE.....	52
5.5.2 VELOCITY PROFILE GENERATION.....	53
5.6. MODEL PREDICTIVE CONTROLLER	53
5.6.1 OBJECTIVE FUNCTION	54
5.6.2 SYSTEM DYNAMICS AND LINEARIZATION.....	55
5.6.3 CONSTRAINTS.....	57
5.7. NETWORK ARCHITECTURE AND HYPERPARAMETERS.....	59
5.8. TRAINING RESULTS	60
5.9. TESTING.....	61
6. DESIGN AND IMPLEMENTATION OF AN END-TO-END APPROACH FOR A COLLISION AVOIDANCE SYSTEM	64
6.1. PROPOSED FRAMEWORK.....	64
6.2. SCOPE AND LIMITATIONS	65
6.3. STATE SPACE	66
6.4. ACTION SPACE	66
6.5. NETWORK ARCHITECTURE AND HYPERPARAMETERS.....	67
6.6. TRAINING RESULTS	68

6.7.	TESTING.....	68
7.	CONCLUSION AND FUTURE DIRECTIONS	72
7.1.	OBSTACLES AND LIMITATIONS	72
7.2.	FUTURE ENHANCEMENTS AND RESEARCH DIRECTIONS	72
7.3.	CONTRIBUTIONS TO THE FIELD.....	73
7.4.	CLOSING REMARKS	74
8.	REFERENCES.....	75
9.	LIST OF FIGURES	81
10.	LIST OF TABLES.....	82
11.	APPENDIX	83
11.1.	LINEAR MPC IMPLEMENTATION	83
11.2.	CODE FOR LINEARIZATION	84

1. INTRODUCTION

Over the past 30 years, research in autonomous vehicles has increased due to advancements in technology, particularly in sensing and computing technology. Self-driving cars promise safer roads by reducing accidents caused by human error, potentially leading to fewer casualties and lower healthcare costs [1].

Furthermore, autonomous vehicles can improve mobility for those who can't drive and make travel more productive and less stressful for commuters. They can also be programmed for fuel efficiency, reducing emissions and benefiting the environment. Beyond personal use, autonomous trucks and delivery vehicles could transform logistics, cutting costs and boosting efficiency [2]. Despite the numerous advantages, how do we make sure these cars can avoid accidents and keep us safe in real-life situations?

Research in autonomous vehicles is currently growing fast. In the hardware domain, a key focus is sensor technology, including cameras, lidar, radar, and ultrasonic sensors. Researchers are dedicated to improving the accuracy, reliability, and cost-effectiveness of these sensors. Equally important is the development of adaptable control systems capable of handling diverse driving scenarios. On the software side, advancements in perception and localization are critical. This involves processing sensor data to identify and track objects and improve vehicle's environmental awareness. Additionally, planning and decision-making algorithms are essential to enable self-driving vehicles to navigate safely, predict the behavior of other road users, and respond to unexpected circumstances [3].

Just as important, safety and validation are primary concerns, with researchers developing methods to ensure the reliability and safety of autonomous systems. Furthermore, the ethical and regulatory dimensions of autonomous vehicles are receiving increased attention, with researchers delving into the societal, economic, legal, and policy implications [3].

Several Advanced Driver Assistance Systems (ADAS) were developed to protect road users. These systems can be only as a warning for the driver or actively brake and steer to avoid an expected collision. A collision avoidance system is designed to detect and respond to potential collisions with other vehicles, pedestrians, or obstacles, taking corrective actions to avoid accidents or mitigate their severity. Collision avoidance stands as a critical concern within the scope of autonomous vehicles, given that a significant proportion of

accidents arise from human errors such as delayed reactions, miscalculations in collision timing, and misjudgments when overtaking other vehicles [4].

Among these systems, the Autonomous Emergency Braking (AEB) system is programmed to reduce vehicle speed when detecting a potential collision. Additionally, the Forward Collision Warning (FCW) system notifies the driver when a potential collision is detected. Furthermore, the Autonomous Emergency Steering (AES) system assists the driver in changing the vehicle's trajectory, this system can autonomously manipulate the steering wheel to avert a collision [3].

The collision avoidance system can be either an AEB, an AES, or even a combination of both. Braking-only systems have a significant limitation: they are unable to prevent collisions when the distance to an obstacle is less than the required stopping distance, brake-and-steer systems aim to address this issue by initiating a lane change or a swerve when they detect that only applying the brakes in a straight line won't avoid the collision [5]. Figure 1 shows a scenario where the obstacle (pedestrian) is occluded and cannot be seen from a safe distance, making it challenging for the AEB system to function effectively, where the AES system would have been beneficial.

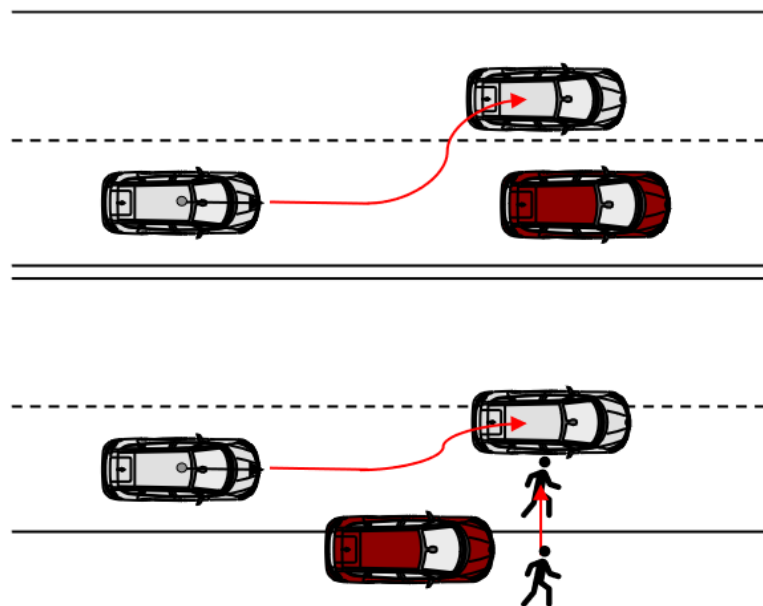


Figure 1: Use cases in which an AES system can mitigate dangerous scenarios where an AEB system may fail [6].

In the context of a collision avoidance maneuver, where the vehicle needs to utilize its maximum capabilities in both longitudinal and lateral directions, the planning of optimal trajectories plays an essential role. This trajectory planning is crucial as it can guarantee the safe operation of an autonomous vehicle [7].

This research aims to perform a comprehensive literature review to assess the current state of collision avoidance systems in autonomous vehicles and to analyze global legal and regulatory frameworks. Based on the insights from the literature review, it aims to define a simulation environment and develop an algorithm for collision avoidance, followed by a test and evaluation of the established system, employing pertinent metrics regarding collision prevention and analyzing the obtained results to draw conclusions regarding the system's performance and limitations.

2. STATE-OF-THE-ART IN COLLISION AVOIDANCE SYSTEMS FOR AUTONOMOUS VEHICLES

In this literature review, several approaches to collision avoidance systems will be explored, from classical approaches to cutting-edge techniques, while paying attention to the recent advancements in this field by summarizing relevant studies, frameworks, and technologies as well as evaluating the existing state-of-the-art in these systems.

Among the collision avoidance systems, the Autonomous Emergency Braking (AEB) system is programmed to reduce vehicle speed when detecting a potential collision. Additionally, the Forward Collision Warning (FCW) system notifies the driver when a potential collision is detected, an example of these system working together can be seen on Figure 2.

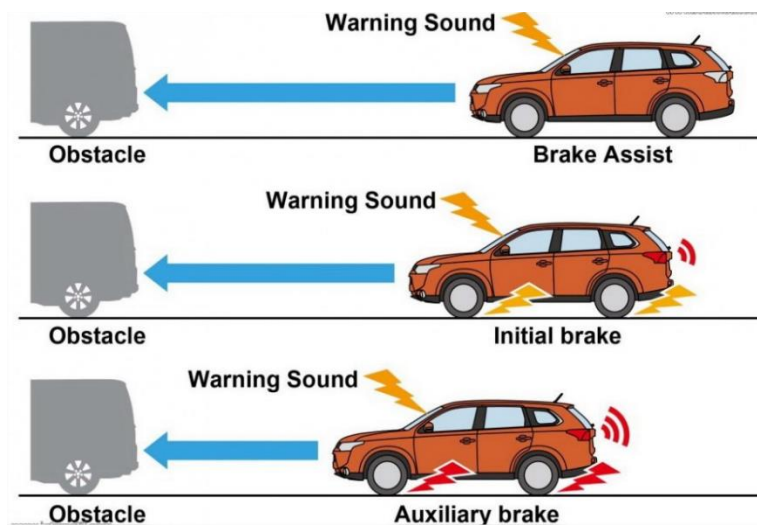
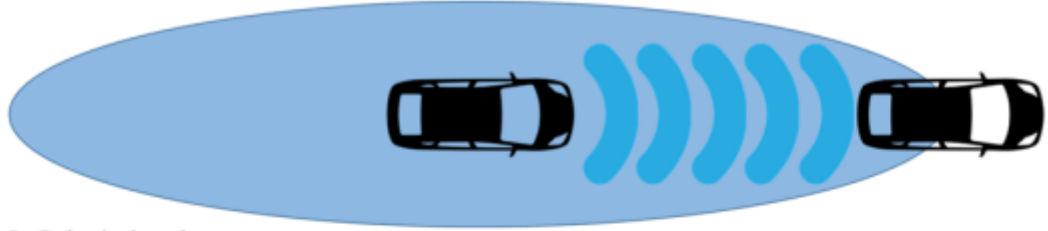


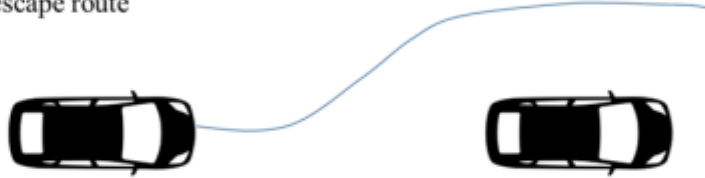
Figure 2: Example of FCW and AEB [8]

Furthermore, the Autonomous Emergency Steering (AES) system assists the driver in changing the vehicle's trajectory, this system can autonomously manipulate the steering wheel to avert a collision, an example of this system can be seen on Figure 3.

1. Detecting surrounding information for obstacle.



2. Calculating the escape route



3. Tracking the escape route trajectory.



Figure 3: Algorithm flow of collision avoidance system by applying AES [9].

A collision avoidance system can be divided into three subproblems. The perception module, the motion planning module, and the control module, as can be seen on Figure 4.

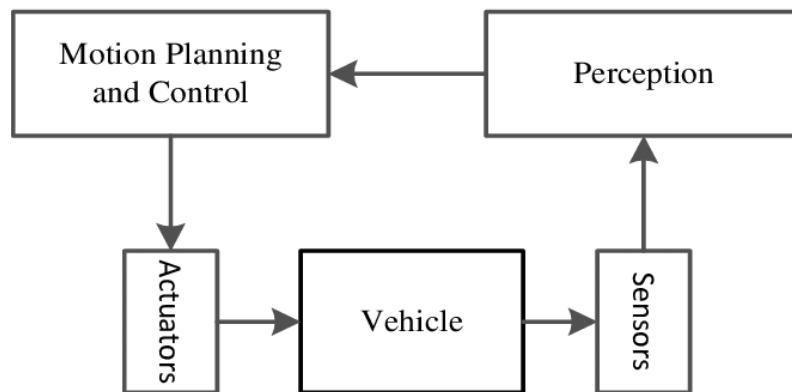


Figure 4: High-level diagram of the control system of a collision avoidance system [10].

The perception module is composed of sensors, that can be camera, lidar, radar, or a combination of them and is responsible to determine whether the emergency system is enabled or not. A collision avoidance system is triggered once a specific threshold is

reached as determined by the predictor, at which point the safety controller is enabled [11]. Various indicators, including time to collision (TTC) [12], time-to-evade (TTE) [6], dynamic safety distance [13], and stochastic reachable set threat assessment [14], may be employed within the predictor to evaluate the risk of collision. Once the safe controller is enabled the challenge represents a subset of motion planning and control. The planning module is responsible for generating a vehicle driving path. The control module is responsible for driving the vehicle [15]. In contrast to conventional motion planning challenges, this scenario involves a more complex environment and demands quicker reaction times. This research and literature review will explore motion planning methods as well as control techniques, focusing on systems developed to avoid collision in emergencies.

Motion planning algorithms can be divided into two main categories: classical and learning algorithms. Classical methods include graph search algorithms, curve interpolation algorithms, sampling-based approaches, artificial potential field methods, and numerical optimization algorithms. The classical motion planning approaches usually follow a pipeline framework (as depicted in Figure 5), where the tasks are divided into perception, localization, planning, and control. This method is widely used in the industry, due to its interpretability, allowing the identification of defective modules when unexpected system behavior occurs. The drawback of this method is that individual sub-modules may not be ideal for all situations, posing a challenge to the generalization of the system. In addition, it requires substantial computational resources and numerous manual heuristics functions that can compromise the robustness and real-time capabilities of this method [16].

In contrast, machine learning solutions address these drawbacks by training agents to generate viable paths. These methods can be integrated within the pipeline framework or employ an end-to-end framework, as shown in Figure 5. The end-to-end framework optimizes the entire driving process, from raw perception to control signals, as a single machine learning task. Key models for this approach include imitation learning, where the model learns from past data, and reinforcement learning, where the model develops a policy through trial and error. While this framework enhances generalization and offers superior robustness and real-time capabilities compared to the pipeline approach, it faces challenges in terms of interpretability. The absence of intermediate outputs makes it harder to trace errors and explain the model's choice of control commands or trajectories [17].

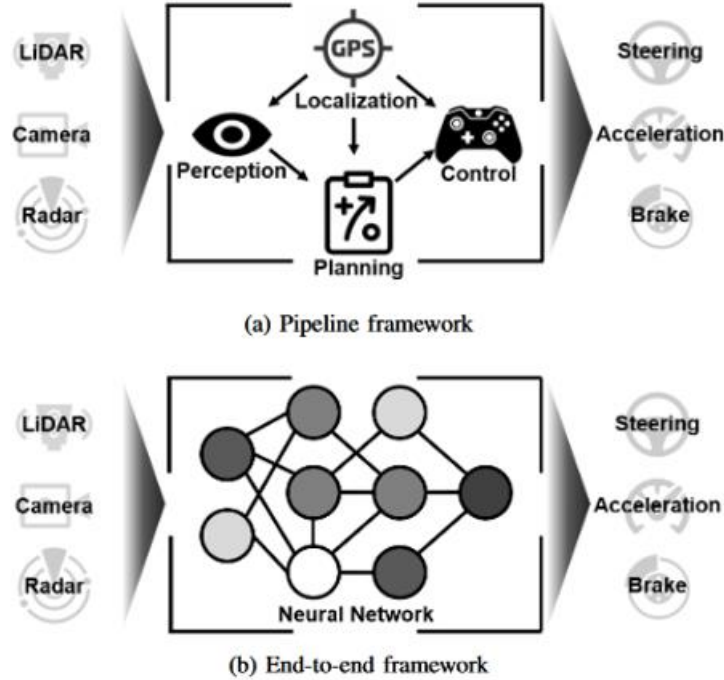


Figure 5: Pipeline and end-to-end frameworks [17].

The classical and learning-based approaches have different strengths and weaknesses, which can significantly impact their effectiveness in various scenarios. This research will explore some key aspects, such as adaptability, computational complexity, real-time capabilities, and interpretability, aspects that are important when designing and implementing collision avoidance systems.

The subsequent subchapters will be structured into two main sections: the classical approaches and the learning approaches. Both methods will be explained, and the existing literature will be discussed as well as its applicability regarding emergency collision avoidance systems.

2.1. CLASSICAL APPROACHES

When discussing the collision avoidance system within the pipeline framework, we are specifically addressing the planning and control subsystem. As discussed before this system differs from conventional motion planning systems, due to its involvement in a more complex environment and its requirement for faster reaction times.

According to [15], vehicle motion planning consists of two stages:

- Path planning: generating a geometric path connecting the vehicle's initial and final points to create a route in the environment and generate a feasible and collision-free geometric path for the vehicle.
- Trajectory planning: determines the vehicle's speed profile along the geometric path generated during path planning. It follows path planning and involves deciding the optimal speed for each point along the path.

In practical engineering applications, motion planning is often structured into a two-stage computational approach, starting with path planning to generate the geometric trajectory, followed by the consideration of speed profiles along this planned trajectory[18].

The control system is responsible for tracking the trajectory provided by the planning system. It does so by appropriately adjusting the final control elements (throttle, brake, and steering) of the vehicle [19]. The two main objectives of motion control are longitudinal and lateral control. Longitudinal control in vehicle control refers to the management of the accelerator and brake to maintain a specific speed, whereas lateral control involves using the steering system to guide the vehicle along a predetermined path [15].

The motion planning algorithms, within the classical approaches, can be categorized as graph search algorithm, curve interpolation algorithm, sampling-based approach, artificial potential field method, and numerical optimization [15]. Each one of them has its own set of advantages and limitations, which will be explored further in the context of their feasibility for emergency collision avoidance systems.

2.1.1 GRAPH SEARCH ALGORITHM

Graph search is a computational process used to explore and navigate networks, or graphs, consisting of interconnected nodes (vertices) and edges (links). It involves finding a path or sequence of nodes within the graph, connecting a specified start node (initial vehicle's position) to a goal node (desired vehicle's position).

The algorithm follows a two-stage process to find paths that satisfy driving conditions: it first constructs a graph and then searches for pathways within this graph. As stated in [15], the main approaches for graph construction are the visual graph method [20], and the Voronoi graph method [21]. Then, the shortest connection defined by a suitably chosen metric is searched along the graph via some heuristics. A* [22] is a widely used pathfinding algorithm. It is designed to find the shortest path from a starting point to a goal node in a graph, where each edge has an associated cost or distance. A* combines the benefits of two other algorithms, Dijkstra's algorithm [23] and Breadth-First Search, by considering both the actual cost to reach a node from the start and a heuristic estimate of the cost to reach the goal. This combination makes A* efficient and guarantees finding the optimal path while exploring a minimal number of nodes in the graph.

To manage complexity and improve performance several extensions of the A* algorithm have been employed in path planning [3]. Likhachev [24] and Ferguson [25], proposed an anytime, replanning search algorithm known as Anytime Dynamic A*, this method quickly generates an initial, sub-optimal plan for the vehicle and then improves the quality of this solution during the available deliberation time. Montemerlo [26], proposed the Hybrid A* algorithm, which is employed to link a continuous state to the discrete representation, resulting in a more feasible and smoother path to follow.

The graph search method is widely used on Unmanned Aerial Vehicles (UAVs) as can be seen in [27] and [28]. Research also indicates its use in speed planning [29] as well as less complex scenarios such as overtaking maneuvers [30]. Although the formulation of graph search-based methods makes it easy to deal with collision avoidance, vehicle dynamics considerations are hard to incorporate and these can be computationally expensive [3], therefore, this method is not commonly used for emergency systems.

2.1.2 CURVE INTERPOLATION ALGORITHM

A curve interpolation algorithm is a method used for generating a smooth, continuous path between discrete waypoints or key positions as well as a feasible trajectory for autonomous vehicles to follow, ensuring that the vehicle can smoothly navigate through complex environments and avoid obstacles effectively. Figure 6 shows an example of curve interpolation algorithm using a Bézier curve to avoid obstacles.

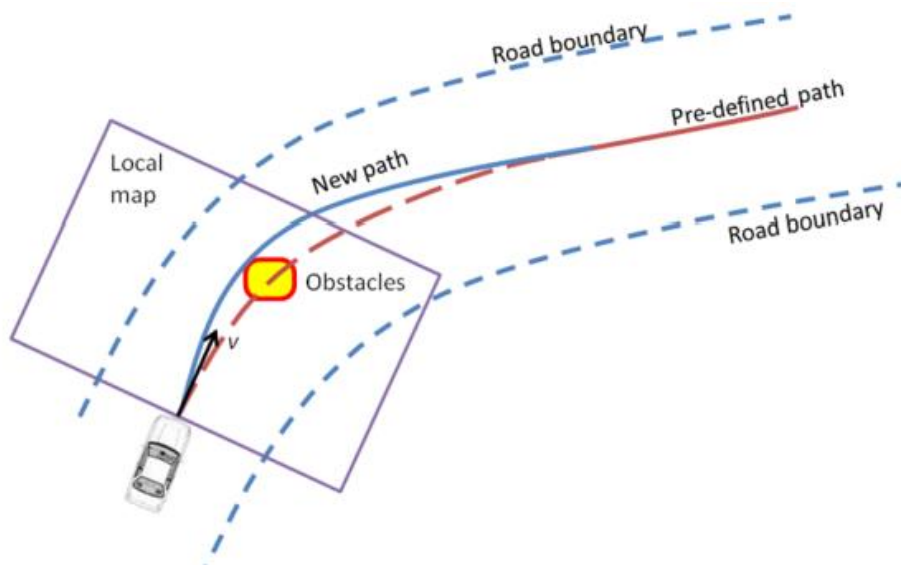


Figure 6: Bézier curve interpolation for obstacle avoidance [31].

The curve interpolation method is easily traceable and it has a low computational cost [15]. However, within this approach, the capacity to account for vehicle dynamics is restricted to incorporating maximal steering angles and geometric acceleration constraints [32]. As a result, the dynamic feasibility of the generated path is not assured and requires subsequent evaluation [33], which might be time-consuming when managing obstacles in real-time [34]. Various curve methods are employed in collision avoidance systems, each with distinct characteristics:

1. **Lines and Circles:** This method is known for its low computational cost and simplicity in implementation. It ensures the shortest path for car-like vehicles. However, it results in non-continuous, jerky transitions between path segments, making it less comfortable. This method is constructed by interpolating known way points with straight and circular shapes [34].

2. **Clothoid Curves:** Clothoid curves offer smooth transitions between straight and curved segments by maintaining a linear change in curvature along the path. They are used in highway and road design and are suitable for local planning [35], [36]. However, the computation can be time-consuming due to the integrals defining the curve, and the curvature behavior is continuous but not entirely smooth. Like other methods, clothoid curve planning depends on global waypoints [34]. Lin et al. [37] developed an emergency avoidance system by combining Adaptive Potential Field and Clothoid curves, first, he uses the PF distribution to find the obstacle-free path considering the possible motion range of the obstacle, then he used the clothoid curve to fit the path under vehicle dynamics constraints to generate a smooth emergency avoidance path.
3. **Polynomial Curves:** Polynomial curves are known for their ability to meet constraints at interpolated points, such as position, angle, and curvature. They can provide both longitudinal and lateral constraints by determining coefficients based on desired values at the start and end of a segment [34]. These curves offer low computational cost and support continuous concatenation for smoother transitions, but their degree, often 4th or higher, can complicate coefficient calculations [38]. Yuan et al. [39] proposed a pedestrian collision avoidance system considering that only braking collision avoidance is not feasible, first, he predicts the direction of the pedestrian movement and plans the path using the cubic polynomial method.
4. **Bézier Curves:** Bézier curves rely on control points to define their shape, making them computationally efficient and easy to manipulate. They support continuous concatenation for comfort but may lose malleability as the curve degree increases, leading to longer computation times [34]. Different degrees of Bézier curves have been implemented in various automated vehicle scenarios, offering modularity and adaptability. Han et al. [31] proposed an anti-collision system based on laser scanner data, consisting of estimating the trajectories and behavior of surrounding objects, and a Bézier curve-based path planner. Zheng et al. [40] combine the Artificial Potential Field method with several specific properties of the Bézier curve to generate a collision-free path.

5. **Spline Curves:** Spline curves can include polynomial curves, b-splines, and clothoid curves divided into sub-intervals with smooth junctions. They offer a balance between computational cost and continuity, providing a general and continuous curvature path controlled by different knots. While they are suitable for various scenarios, their results may not always be optimal from the perspective of road fitness and curvature minimization [34]. Hoek et al. [18] presented a method that uses B-splines to generate cooperative trajectories. Wang et al. [41] proposed an algorithm that can generate collision-free obstacle avoidance paths while meeting the real-time requirements and dynamic constraints of obstacle avoidance scenarios. First, a dynamic risk identification model based on the support vector machine is proposed. Then, the B-spline algorithm model is improved based on the risk-identification model.

2.1.3 SAMPLING-BASED APPROACH

A Sampling-Based Approach for collision avoidance systems is a path-planning strategy that addresses high-dimensional configuration or state spaces, where deterministic methods may struggle to provide timely solutions. It primarily involves random sampling within the configuration or state space to establish connectivity [42]. The most widely used techniques in robotics for this approach are the Probabilistic Roadmap Method (PRM) and the Rapidly-exploring Random Tree (RRT) [34].

RRT algorithms have been widely adopted for path planning in complex spaces. They initiate from the vehicle's starting state and create a tree structure by branching out based on dynamic constraints. As the tree reaches the desired end state, the corresponding branches define the path [3]. RRT conducts a random search through the navigation area, which facilitates consideration of non-holonomic constraints like turning radius and vehicle momentum. Although RRT can provide fast solutions, the resulting path may not be optimal, leading to jerky and non-curvature continuous trajectories [34]. An improved version, RRT*, offers convergence to an optimal solution but retains these disadvantages [43]. To address the dynamic feasibility and real-time need for urban environments, Kuwata et al. [44] proposed the CL-RRT algorithm, using closed-loop prediction in the framework of RRT. The combination of stabilizing controller and forward simulation has enabled the application of CL-RRT to vehicles with complex, nonlinear, and unstable

dynamics. Yang and Lin [45] developed and improved rapidly exploring random tree algorithms for static obstacle avoidance in autonomous vehicles.

The RRT algorithm can provide a fast solution in multi-dimensional systems. It is suitable for global and local planning [34]. While the resulting trajectory may not always offer continuous and smooth transitions, it prioritizes efficiency over comfort, which can be crucial in emergency systems.

2.1.4 ARTIFICIAL POTENTIAL FIELD METHOD

The artificial potential field method models vehicle movement in an environment influenced by virtual force fields. In this method, the environment is represented as attractive forces, guiding the vehicle toward desired motions, and repulsive forces, which help the vehicle avoid obstacles such as road borders, lane markings, and other vehicles, an example of how the artificial potential function works can be seen on Figure 7. The main advantage of APF is its reactivity to the dynamic evolution of the scene, allowing it to adapt to changing situations. The motion of the ego vehicle is guided by the resultant vector of these forces, eliminating the need for explicit space decomposition. The resolution of the resultant vector can be achieved through gradient descent methods or by applying Newton's second law based on a vehicle model [36].

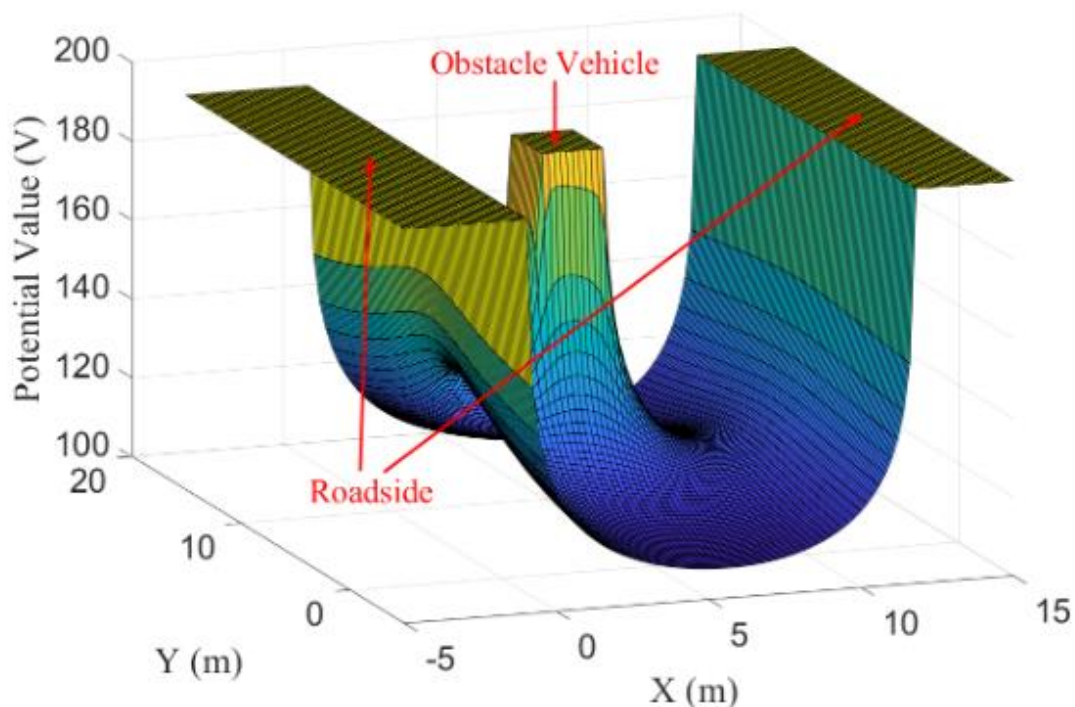


Figure 7: Potential Field in a Highway [11].

The APF concept was first introduced to real-time mobile robotics [46] and it has been adapted for scenarios involving lane modeling, road representation, obstacle avoidance, and desired speed, among others [47]. Shang et al. [11] proposed a new artificial potential function inspired by line charge as the cost function for a model predictive controller. Wang et al. [41] analyze the characteristic of human drivers to define a safety model of obstacle avoidance. Based on the safety model, the artificial potential field method is improved and the repulsive field range of obstacles is built. Finally, based on the improved artificial potential field, a collision-free path for autonomous driving vehicles is generated. On [48] the motion planning for collision avoidance is formulated using an artificial potential field approach based on the risk perception of human driver. Aiming to solve the local-minima problem, Qingfen et al. [49] propose a combination of APF and Reinforcement Learning, as a first step “black holes” are created on the target destination of the potential field and the RL agent learns how to jump out of the local-stable-point and achieves the target based on the potential field information.

APF Implementations have been successful in collision avoidance systems for automated vehicles. However, some limitations include a tendency to become stuck in local minima and challenges in reaching the target point when influenced by repulsive forces. Nonetheless, the APF method remains a valuable tool for path planning in dynamic and uncertain environments [15].

2.1.5 NUMERICAL OPTIMIZATION ALGORITHM

These optimization techniques can serve as the refinement of precomputed trajectories, usually based on some geometric method, and evaluated through a value function on how well the vehicle can travel along the path, using methods like Model Predictive Control (MPC) to enhance safety and path quality [3]. Furthermore, numerical optimization can play a role in computing viable motion paths while adhering to dynamic constraints such as position, velocity, acceleration, and jerk. This is accomplished by minimizing a cost function that considers these constraints [34]. This approach can generate continuous and secure motion paths. However, the computational complexity of these methods is a pressing challenge, often requiring trade-offs to ensure real-time decision-making. Linear Parameter Varying (LPV) models and modern MPC techniques can provide near-optimal

solutions, but this may come at the cost of high computational requirements, often challenging real-time applications [3].

Voesten [7] combined a nonlinear single-track vehicle model with a nonlinear combined slip tire model to generate the trajectory and a IPOPT NLP solver in combination with the ICLOCS2 software to solve the optimization problem for a collision avoidance maneuver within and beyond stable limits of handling (drifting). Aiming to improve computation efficiency while maintaining high-performance capabilities, Alcalá et al. [50] proposed a transformation of the non-linear vehicle equations into a Linear Parameter Varying (LPV) representation. The approach to formulating the Model Predictive Planning (MPP) problem involves the convexification of non-linear objective functions into a linear-quadratic format. Then, this LPV vehicle model is used to predict trajectories within a defined horizon including the ability of avoiding obstacles in a simple way and with reduced computational cost. In [51], Anistratov focuses on autonomous avoidance maneuvers in critical driving situations, particularly using dynamic optimization, the contributions include investigating new optimization criteria, studying recovery behavior after passing obstacles, segmenting maneuvers, addressing multiple obstacles, and developing a real-time controller.

In conclusion, classical approaches are a fundamental tool in the development of collision avoidance systems. These methods generate safe and efficient motion trajectories, considering dynamic constraints and optimizing cost functions. While challenges like computational complexity and real-time processing remain, ongoing research and advancements aim to solve this problem, ultimately leading to more reliable and responsive collision avoidance systems in autonomous driving.

2.2. LEARNING APPROACHES

Machine learning solutions offer a way to address the challenges of computational complexity and real-time processing thanks to their high performance in learning, adaptation and generalization, as well as the fact that the computationally demanding tasks can be carried out offline [3].

Most machine learning solutions for autonomous vehicles predominantly adopt an end-to-end framework, where the model takes raw data inputs and directly determines control outputs. Key models within this paradigm include imitation learning, which learns from

historical data, and reinforcement learning, which develops control policies through trial and error. As discussed before, this framework enhances generalization, robustness, and real-time capabilities when compared to the pipeline approach, but it encounters challenges related to interpretability, as the absence of intermediate outputs makes it more challenging to trace errors and explain the model's selection of control commands or trajectories [17].

2.2.1 SUPERVISED LEARNING

The supervised learning models relies on labeled data to train machine learning models. This approach learns to recognize patterns and make informed decisions by observing examples provided during training. Each example includes a description of the environment, vehicle state, and the correct action to take for collision avoidance, which can be steer and/or brake.

One of advantages of supervised learning is its ability to make real-time decisions, since the training is carried out offline. The system can learn from a vast array of scenarios, from simple lane-keeping [52] to complex multi-agent interactions [53]. By capturing patterns in data, the model can identify dangerous situations and execute the appropriate avoidance maneuvers, such as steering, braking, or accelerating.

However, supervised learning also presents challenges, particularly in the need for large and diverse datasets to ensure robust and safe operation [54], since the model learns from past data, it may not encounter every corner situation, thus, a limitation of this approach is the inability to handle unknown or complex scenarios [55]. Furthermore, issues related to model interpretability and the ability to explain why a specific action was chosen are crucial.

Several end-to-end approaches have been developed using supervised learning, a comprehensive study on supervised learning methods for autonomous driving, also referred to as behavior cloning methods, can be found in [56]. Chen and Huang [52] proposed a convolutional neural network that takes raw image frames as input and outputs the steering angles accordingly, they show that the model can achieve a relatively good accuracy and proposes data augmentation to increase performance. Elkholy [57] introduced two convolutional neural network architectures, each with varying numbers of filters in their layers. These networks were trained to process input images by scanning the raw pixels and directly translating them into steering angle commands and speed values. Additionally,

Elkholy's work included the development of an object recognition algorithm capable of detecting objects and determining their distances from the controlled vehicle. This was achieved using a pre-trained single shot detector model, creating a collision warning system.

Jafri and Kala [53] introduce a approach to end-to-end motion planning by modeling and learning specific obstacle avoidance behaviors, particularly in scenarios where dynamic obstacles approach a robot directly. To address this challenge, a deep neural network (DNN) was trained with two distinct obstacle avoidance strategies inspired by human-like obstacle avoidance behavior. The paper adopts behavior-specific training to intensify exposure to rare corner cases, enhancing the robot's learning performance. Comparative evaluations against three state-of-the-art motion planning methods demonstrate the superior performance of the proposed approach.

In the context of motion planning, Hegedüs et al. [58] introduced a hybrid method using artificial neural networks (ANNs). Their approach begins with the presentation of a trajectory planner based on nonlinear optimization, ensuring the vehicle's motion is dynamic feasible. Following this, an artificial neural network is trained through supervised learning to simulate the behavior of the optimization-based planner. The generation of training data occurs offline, eliminating concerns regarding the computational demands of the optimization-based technique. Subsequently, the trained neural network takes the place of the original motion planner during online planning tasks, leading to a significant reduction in computational workload and, consequently, runtime efficiency.

In conclusion, supervised learning models has the capability to recognize patterns and make informed decisions based on labeled data. This approach can adapt to diverse scenarios, from simple lane-keeping to multi-agent interactions. However, it is still relatively unexplored in the context of collision avoidance systems due to the inherent complexity of the problem and also has many challenges regarding the need for extensive and diverse datasets. To solve this problem, reinforcement learning techniques are being widely explored in the context of autonomous vehicles [3].

2.2.2 REINFORCEMENT LEARNING

Collision avoidance systems need to make quick and reliable decisions in dynamic and often unpredictable traffic scenarios. While classical approaches have made significant improvement in this field, the increasing complexity of real-world traffic environments and the need for systems that can adapt and generalize to new situations have increased the interest in reinforcement learning (RL).

Reinforcement learning is a subfield of machine learning where an agent learns to make sequences of decisions by interacting with an environment. Unlike supervised learning, where a model is trained on labeled data, RL operates in scenarios where the agent receives feedback in the form of rewards or punishments, guiding it towards optimal decision-making [59]. In the context of collision avoidance, RL models learn to take actions that maximize the cumulative reward while avoiding collisions and ensuring safe vehicle operation.

Regarding motion planning using deep reinforcement learning, a comprehensive survey can be seen on [60]. In [54], a framework for path tracking is proposed, focusing on mimicking human driving behavior. The approach combines behavior cloning (imitation learning) and deep reinforcement learning (DRL). Initially, behavior cloning is used to initialize the DRL algorithm by considering the experiences of professional drivers. Then, a continuous, deterministic model-free DRL algorithm is adopted to train the model through trial and error. The Actor-Critic structure is applied in the DRL algorithm and the DRL model learns an effective path tracking policy.

Ibrahim et al. [61], explores the Q-learning algorithm to solve the local path planning with obstacle avoidance problem. Q-learning involves two phases: training and application. During training, it demands increasing time investments in proportion to the system's state space. However, during the application phase, Q-learning simplifies into a lookup table, making it suitable for even basic microcontrollers. Two simulations were conducted in different environments. The first demonstrated Q-learning's capability to learn optimal paths in a static environment. The second simulation explored Q-learning's adaptability in randomized, dynamic environments, successfully finding paths to the target when they existed, although the algorithm still found some challenges where the vehicle did not find any feasible path.

Qian et al. [62], tackles the challenge of obstacle avoidance path planning for autonomous vehicles, considering maneuvers like emergency braking and active steering to ensure safety and comfort. This approach employs Deep Q-Network (DQN) to train the vehicle to generate a secure and comfortable obstacle avoidance path, incorporating longitudinal braking safety distance, lateral lane change, and constraints imposed by a Fuzzy-PID controller. The inclusion of an LSTM layer in the network enables learning from a sequence of observations, enhancing adaptability to dynamic environments. A unique reward mechanism is introduced, considering safety distance for different maneuvers (brake deceleration, stop waiting, lateral lane change) to promote optimal obstacle avoidance actions. Finally, the DQN model is simulated and trained using open-source software, CARLA [63], the approach's effectiveness assessed based on collision rates, total rewards, trajectory curves, and vehicle parameters across various traffic scenarios.

Fehér [64], tackles the task of planning optimal trajectories for emergency avoidance maneuvers in highly automated vehicles. The proposed solution involves a hierarchical approach for avoiding moving objects. A reinforcement learning agent is responsible for local planning, while low-level MPC and Stanley controllers manage longitudinal and lateral control. In this architecture, the agent is trained to provide the parameters for a polynomial-based path and velocity profile using a neural network output. The model receives feedback from the low-level controllers to determine reward values and make predictions. The framework was tested in a simulation environment and compared to human drivers' capabilities.

Although some hybrid approaches combine strategic and direct control, as demonstrated in the work of Qian et al. [62], only a limited number of papers explore the integration of a geometric path definition through reinforcement learning, followed by controller-based execution, as seen in the studies by Ronecker and Zhu [59] and Fehér et al. [65]. Currently, state-of-the-art approaches suggest that combining machine learning with modern control solutions in a hierarchical system often yields promising results [3].

Reinforcement learning holds promise for collision avoidance systems due to several factors. First, RL offers adaptability, enabling the system to navigate complex and dynamic traffic scenarios effectively. It can generalize from training data to make real-time decisions in situations not explicitly seen during training. Second, RL can potentially handle multi-agent interactions, where the behavior of other road users impacts collision

avoidance decisions. Lastly, the ability to learn online from interactions with the environment, or self-play, is a significant advantage, as it allows the system to continuously improve its decision-making.

However, one significant drawback is the need for substantial computational resources and the demand for extensive training. These agents require large amounts iterations to learn effectively and encounter the sim-to-real problem, which involves the difficulty of transferring a policy learned in a simulated environment to real-world performance [60]. Furthermore, the interpretability of reinforcement learning models remains an issue, making it difficult to explain why a specific decision was made in a particular situation.

2.3. COMPARATIVE ANALYSIS OF COLLISION AVOIDANCE METHODS FOR AUTONOMOUS VEHICLES

When developing collision avoidance systems for autonomous vehicles, the benefits and limitations of various approaches needs to be considered. Based on the discussion of each method provided in the previous subchapters a comparative analysis will be provided.

Graph Search Algorithms systematically explore possible paths, which is beneficial for environments with predictable patterns. However, it is computationally expensive when dealing with complex environment, potentially compromising their real-time application in dynamic settings where obstacles and traffic conditions change rapidly.

Curve Interpolation Algorithms have smooth transitions in vehicle movement, contributing to comfort and control. They generate paths by connecting a series of waypoints with curves. The limitation lies in their need to additional checks to validate dynamic feasibility.

Sampling-Based Approaches, such as Rapidly-exploring Random Trees (RRT), offer a robust solution in high-dimensional and unpredictable environments. They can navigate around both static and dynamic obstacles. However, the paths produced may not me optimal and require post-processing to ensure they meet all vehicular constraints.

Artificial Potential Fields are simple and have low computational need. They operate by creating a virtual field where obstacles exert repulsive forces and goals exert attractive forces, guiding the vehicle accordingly. While effective for obstacle avoidance, this method can encounter difficulties such as local minima, where the vehicle can be trapped in an area surrounded by obstacles.

Numerical Optimization Algorithms have the ability to produce paths that are both dynamically feasible and safe, adhering closely to the vehicle's capabilities. However, the complexity and computational intensity of these algorithms are a problem for time-critical situations.

Supervised Learning relies on past data to make decisions. It's good for known situations but can struggle with new or unexpected events on the road since it relies on having seen something similar before.

Reinforcement Learning is a trial-and-error learning process, where the system improves over time through interactions with its environment. It's highly adaptable and can theoretically handle complex traffic scenarios with multiple interacting agents. The trade-off, however, comes with the need for extensive training iterations and computational resources, along with the challenge of interpreting the decision-making process.

Table 1 provides a summary of the mentioned methods, providing a comparison of their advantages and disadvantages in terms of real-time performance, vehicle dynamics, computational demands, path optimization, robustness, implementation ease, and other important factors relevant to the development and integration of a collision avoidance system.

Table 1: Advantages and Disadvantages of each mentioned approach.

	Approach	Advantages	Disadvantages
Classical Approaches	Graph Search Algorithms	- Suitable for detailed path planning in multi-dimensional spaces	- Can be computationally expensive and not real-time friendly.
	Curve Interpolation Algorithms	- Low computational cost and simplicity with smooth transitions.	- Limited dynamic feasibility without additional evaluation.
	Sampling-Based Approaches	- Quickly generates paths in complex spaces.	- Paths generated may not be optimal and smooth.
	Artificial Potential Field	- Simple and computationally efficient.	- May get stuck in local minima.

	Numerical Optimization Algorithms	- Generates paths that adhere to vehicle dynamics.	- High computational complexity can be a challenge for real-time processing.
Learning Approaches	Supervised Learning	- Ability to make real-time decisions based on labeled data.	- Requires large and diverse datasets to ensure robustness.
	Reinforcement Learning	- Real time application after training and adaptability.	- Extensive training iterations needed; sim-to-real transfer issues.

2.4. ADVANCED COLLISION AVOIDANCE STRATEGIES FOR AUTONOMOUS VEHICLES

The progression of autonomous vehicle (AV) technologies in the domain of collision avoidance shows an evolution from classical approaches to more modern learning-based methodologies, or even a combination of both in a hybrid approach. In this subchapter some advance approaches for collision avoidance will be explored.

From the classical approaches, particularly the geometric-based approaches, it is worth mentioning the work from van der Ploeg et al. [6], who developed a functional architecture for an Autonomous Emergency Steering (AES) system designed to avoid collisions by steering. The architecture integrates vehicle capability estimation, path planning and replanning, path rejection and ranking, selection, and an evasive trigger mechanism. It employs clothoid-based path planning for dynamic feasibility within the collision-free space. The system is validated through simulation and field tests, demonstrating its ability to plan, adjust, and execute collision avoidance maneuvers in real-time scenarios.

Following the idea of dynamic path planning, Zheng et al. [40] introduced a trajectory planning algorithm that combines quartic Bézier curves with potential field function. This method focuses on generating collision-free trajectories, especially on curving roads where traditional potential field approaches may struggle. Simulated scenarios, including lane-changing, lane-keeping, and collision avoidance, demonstrate the algorithm's effectiveness.

From the sampling-based approaches, Yang and Lin [45] advanced path planning techniques by improving the Rapidly Exploring Random Trees (RRT) algorithm for static obstacle avoidance. Their method integrates path pruning, smoothing, and optimization with geometric collision detection. Followed by an optimizer to select the shortest path among the feasible paths and by low-level controllers to track the path. Simulation results show that the improved RRT algorithm can plan a collision-free, safe path in multiple obstacle environments.

Transitioning towards more adaptive systems, Shang and Eskandarian [11] introduced a Model Predictive Control (MPC) and Artificial Potential Function (APF) based strategy for emergency collision avoidance and mitigation. This approach integrates an artificial potential function inspired by electrical charge distributions to dynamically adjust vehicle trajectories, ensuring collision avoidance or mitigation (when collision avoidance is not possible) by preferentially protecting human-occupied sections of the vehicle. Simulation results shows higher success rates in avoiding collisions and significantly reducing collisions that occur at the driver's position when protection mode is active.

Numerical optimization techniques further advanced the field, with Voesten [7], where he focuses on generating optimal collision-free trajectories for autonomous vehicles during evasive maneuvers, extending beyond the stable limits of handling. It introduces an approach that has a racing-inspired objective functions, such as minimum time or maximum exit velocity, to fully utilize a vehicle's capabilities in both lateral and longitudinal directions. The methodology incorporates a nonlinear single-track vehicle model, tire models adaptable to different road surfaces, and modern optimization frameworks for solving. Simulations validate the effectiveness of generated trajectories in safely navigating emergency scenarios.

Schratter et al. [66], contributed to this evolution by focusing on a pedestrian collision avoidance system for autonomous vehicles, particularly when pedestrians are obscured, such as by parked cars. It combines autonomous emergency braking and evasion maneuvers with a situation awareness planner to optimize driving speed while considering sensor visibility and system capabilities. The system's effectiveness is validated against Euro NCAP scenarios, demonstrating its ability to generate collision-free trajectories in complex urban settings.

The adoption of learning approaches introduced a new dimension to collision avoidance, with Ibrahim et al. [61] exploring the use of Q-Learning for autonomous vehicle

path planning, focusing on obstacle avoidance. This work divides Q-learning into two phases: a training phase, where the model learns to navigate environments with increasing complexity, and an application phase, where the learned strategies are deployed in real-time. The study validates the approach through simulations in static and dynamic environments, demonstrating its capability to plan paths capable to handle collision avoidance and adapt to new scenarios.

Further advancements are illustrated by Zong et al. [67], who explored obstacle avoidance using an improved Deep Deterministic Policy Gradients (DDPG) method for self-driving vehicles. This study underscores the importance of continuous action spaces and the integration of complex sensor inputs to enhance the navigation and decision-making capabilities of autonomous systems. Zong's work demonstrates the practical application of DDPG in environments simulated by TORCS, highlighting the robustness and adaptability of reinforcement learning strategies.

Following the advancements of learning-based approaches, Fehér et al. [64] introduces a reinforcement learning-based method for online trajectory planning for pedestrian avoidance in autonomous vehicles. This method employs a hierarchical structure, where a reinforcement learning agent is tasked with local planning, and a low-level controller handles longitudinal and lateral controls. The approach uses polynomial-based paths and velocity profiles optimized by the agent, which learns from dynamic scenarios to minimize accident risk and maintain vehicle control. The results demonstrated successful avoidance maneuvers in simulated environments, outperforming human drivers in terms of safety and efficiency.

The design of the reward function is perhaps the most crucial part of advancing reinforcement learning in vehicle collision avoidance, as detailed by Zakaria et al. [68], where the design of multiple reward functions was found to impact the performance of systems in dynamic environments. As emphasized by them, the design of the reward function plays an important role in the reinforcement learning process, determining the effectiveness of the learning outcomes in complex scenarios.

In conclusion, the development of collision avoidance technologies for autonomous vehicles shows a trajectory from geometric, sampling-based, and potential field methods to the modern numerical optimization algorithms and machine learning. The field continues to evolve to more intelligent and adaptive systems that are also capable of ensuring higher levels of safety and efficiency in complex driving environments.

3. LEGAL AND REGULATORY FRAMEWORK FOR COLLISION AVOIDANCE SYSTEMS.

The development and implementation of Collision Avoidance Systems represent an important advancement for road safety and transportation efficiency. However, as autonomous driving technologies continue to grow, so does the need for a robust regulatory and legal framework to ensure the safe and responsible use of these systems on public roads. In this chapter, the legal framework will be discussed, considering international, national, and regional laws, regulations, standards, and guidelines that manages these systems.

At the international level, organizations such as the United Nations Economic Commission for Europe (UNECE) and EuroNCAP have taken steps to address the regulatory challenges of AVs. Euro NCAP, the European New Car Assessment Programme, is an organization committed to improve vehicle safety standards across Europe. This organization has a five-star rating system that ranks the safety of vehicles, for the benefit of consumers and vehicle fleet managers. Its primary objectives include promoting awareness of the benefits of automated driving technologies and setting standards in all vehicle segments [69].

UN Regulations provides material related to safety and environmental considerations for vehicles, their systems, parts, and equipment. These regulations consist of performance-based testing requirements and administrative procedures [70]. The regulations related to automated systems are currently in early stages, being the first regulation for automated driving functionality of Level 3 implemented on 2021, the UN Regulation No. 157 – Automated Lane Keeping System [71]. While these regulations are relatively recent, some of them are relevant to collision avoidance systems:

1. **UN Regulation No. 152 - Advanced Emergency Braking System (AEBS):** this regulation discusses the specifications for AEB systems, which automatically apply the brakes to prevent or mitigate collisions [72].
2. **UN Regulation No. 79 - Steering Equipment:** This regulation sets standards for steering systems, while also mentioning Autonomous Emergency Steering systems [73].

3.1. EURO NCAP 2025 ROADMAP

According to EuroNCAP 2025 [69] approximately 20 percent of Killed and Seriously Injured (KSI) incidents arise from loss of control or lane/road departure, while frontal collisions with a small overlap constitute around 15 percent of all car accidents and 25 percent of those involving frontal collisions. Vulnerable road users account for a significant 36 percent of KSI cases. AES has the potential to significantly reduce these statistics by preventing or mitigating crashes. While AES technology has been technically feasible, it is expected to gain more prominence in the coming years, facilitated by the UN Regulation 79.

EuroNCAP provides technical bulleting regarding testing procedures for these systems. The Technical Bulletin (TB 037) [74] outlines the requirements and testing procedures for Emergency Steering Support (ESS) Systems. ESS functions are designed to complement Autonomous Emergency Braking (AEB) systems by assisting drivers in steering around hazards and avoiding collisions. Here's a summary of the key points and requirements covered in the bulletin:

- ESS functions are considered supplementary to AEB systems.
- ESS systems must be default ON.
- Driver initiation and steering wheel input are required to activate ESS.
- The system should support the driver's steering inputs to avoid collisions and return to the adjacent lane.
- Forward collision warnings must be issued.
- The system should work symmetrically for both left-hand and right-hand drive versions.
- Applicable speeds for ESS are 50 km/h and above.
- Systems supporting speeds beyond 80 km/h may qualify for additional rewards.
- A visual warning for the driver is recommended but not required.
- ESS systems qualify for rewards if they avoid impact with the target and stay within the adjacent lane for at least 2 seconds after the Time-to-Collision (TTC) reaches zero.

The bulletin also provides some test scenarios to test these systems. The scenarios presented can be seen on Figure 8 and Figure 9. The Car-to-Car Rear Stationary (CCRs) (Figure 8) is a scenario where a collision happens in which a vehicle travels forwards towards another stationary vehicle and the frontal structure of the vehicle strikes the rear structure of the other [75]. The Car-to-Pedestrian Longitudinal Adult 25% (CPLA-25) scenario (Figure 9) is a scenario where there is a collision in which a vehicle travels forwards towards an adult pedestrian walking in the same direction in front of the vehicle where the vehicle strikes the pedestrian at 25% of the vehicle's width when no braking action is applied or an evasive steering action is initiated after an FCW [76].

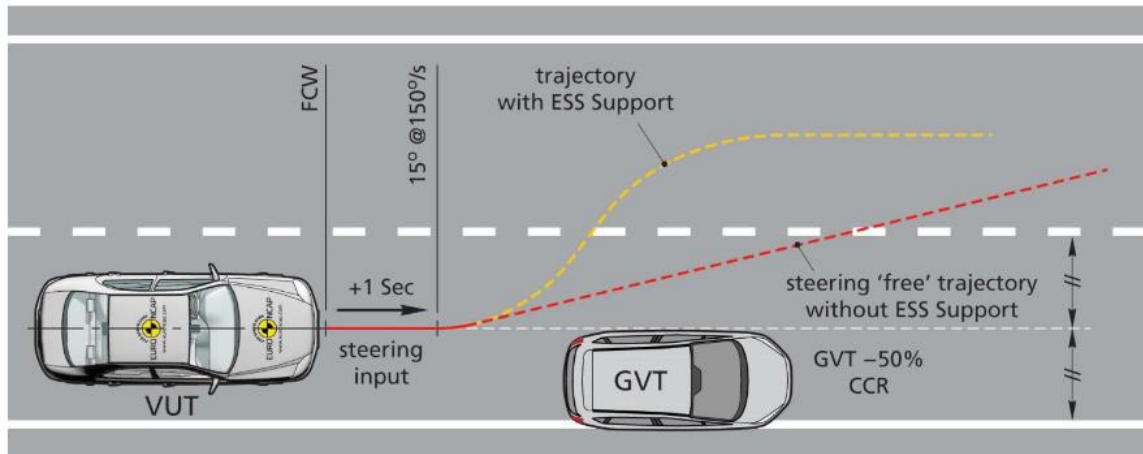


Figure 8: ESS Test diagram using CCRs -50 scenario [74].

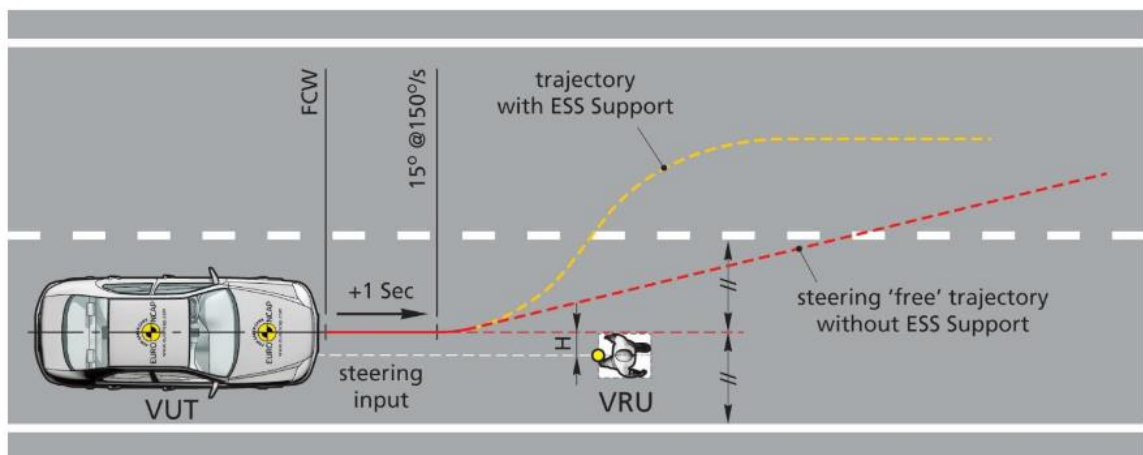


Figure 9: ESS Test diagram using CPLA25 scenario [74].

Additional collision avoidance scenarios from the EuroNCAP documentation [75] are outlined below for further reference and testing of autonomous systems. These scenarios are critical in simulating varied and complex driving situations to ensure that collision avoidance systems can handle different types of collisions effectively:

- **Car-to-Car Rear Stationary (CCRs):** A scenario where one vehicle collides with the rear of a stationary vehicle.
- **Car-to-Car Rear Moving (CCRm):** Involves a collision where one vehicle hits the rear of another vehicle moving at a constant speed.
- **Car-to-Car Rear Braking (CCRb):** This scenario sees a vehicle hitting the rear of another vehicle that is moving at a constant speed but then decelerates.
- **Car-to-Car Front Turn-Across-Path (CCFtap):** Occurs when a vehicle turns across the path of an oncoming vehicle, resulting in a frontal collision.
- **Car-to-Car Crossing Straight Crossing Path (CCCscp):** A vehicle moves straight across a junction and collides with another vehicle crossing perpendicularly.
- **Car-to-Car Front Head-On Straight (CCFhos):** Involves a head-on collision where one vehicle drifts into the opposing lane and strikes another vehicle traveling straight within its lane.
- **Car-to-Car Front Head-On Lane Change (CCFhol):** A scenario where a vehicle changes lanes to overtake and collides head-on with another vehicle traveling in the opposite direction.

Additional EuroNCAP collision avoidance scenarios involving pedestrians and bicyclists include:

- **Car-to-Pedestrian scenarios:** covers incidents pedestrians including various situations where vehicles may encounter these vulnerable road users under different conditions, such as crossing from nearside or farside, and both adults and children are considered.
- **Car-to-Bicyclist scenarios:** cover incidents where vehicles collide with bicyclists crossing the path or traveling in the same direction.

These scenarios are designed to test the robustness and responsiveness of collision avoidance systems under varied traffic conditions and accident risks, enhancing their reliability and safety in real-world applications.

3.2. UN REGULATION NO. 152 - ADVANCED EMERGENCY BRAKING SYSTEM (AEBS)

This regulation provides standards for Advanced Emergency Braking Systems (AEBS) [72] installed in motor vehicles primarily used in urban driving conditions. The AEBS is designed to automatically detect potential forward collisions, warn the driver, and activate the vehicle's braking system to prevent or reduce the severity of a collision if the driver doesn't respond. The system should not compromise the vehicle's safety in case of a system failure. Drivers can always override the system with manual control.

The regulation considers that not all traffic and environmental conditions can be covered in the process. The AEBS should not trigger false warnings or braking that might encourage drivers to deactivate the system. It's an "if-fitted" regulation, meaning it doesn't mandate the installation of AEBS but sets standards for those equipped with it. Here are some important points covered on this regulation:

- Vehicles with AEBS must meet performance and safety requirements.
- The system must not be affected by magnetic or electrical fields.
- Safety aspects of electronic control systems must comply.
- The system must provide specific warnings to the driver.
- Emergency braking must significantly reduce vehicle speed.
- The system should minimize false warnings and braking actions.
- Regulations cover car-to-car and car-to-pedestrian scenarios.
- They detail the requirements for collision warnings and emergency braking in these scenarios.
- Speed ranges for system operation are specified.
- The system's effectiveness and control strategy are detailed for different conditions, including dry roads, laden or unladen vehicles, and various illumination conditions.

In summary, this regulation sets the standards for Advanced Emergency Braking Systems in vehicles, ensuring their effectiveness in several scenarios while maintaining safety and minimizing false alerts. It focuses on preventing or reducing the severity of collisions with other vehicles and pedestrians, especially in urban driving conditions.

3.3. UN REGULATION NO. 79 - STEERING EQUIPMENT

This regulation [73] focuses on the layout and performance of steering systems installed in vehicles used on the road. This regulation has evolved to accommodate advanced steering technologies.

ADAS systems related to the steering equipment consists of systems that assist the driver in steering while still keeping the driver in primary control of the vehicle. The regulation covers some aspects, including the introduction of Advanced Driver Assistance Steering Systems (ADAS) and functions related to steering control, such as Automatically Commanded Steering Functions (ACSF), Corrective Steering Functions (CSF), and Emergency Steering Functions (ESF).

3.3.1 AUTOMATICALLY COMMANDED STEERING FUNCTION (ACSF)

ACSF functions are initiated based on signals from onboard systems and may include low-speed maneuvering, lane guidance, lane-keeping, and other steering interventions. ACSF is further categorized into subtypes:

- **Category A:** This function operates at low speeds, typically no greater than 10 km/h. It assists the driver in low-speed or parking maneuvers.
- **Category B1:** This function assists the driver in keeping the vehicle within the chosen lane by influencing the lateral movement of the vehicle.
- **Category B2:** Similar to Category B1, but it can operate for extended periods without further driver input.
- **Category C:** This function is initiated and activated by the driver, and it can perform a single lateral maneuver (e.g., a lane change) when commanded by the driver.

- **Category D:** Similar to Category C, it indicates the possibility of a single lateral maneuver but only performs it following confirmation by the driver.
- **Category E:** This function is initiated and activated by the driver and can continuously determine the possibility of a maneuver (e.g., a lane change) and complete these maneuvers for extended periods without further driver command/confirmation.

3.3.2 CORRECTIVE STEERING FUNCTION (CSF)

CSF is another control function within the electronic control system. It is responsible for making limited adjustments to the steering angle of one or more wheels based on automatic evaluation of signals initiated on-board the vehicle. These adjustments are made for various purposes, including compensating for unexpected changes in side force, improving vehicle stability, or correcting lane departures.

3.3.3 EMERGENCY STEERING FUNCTION (ESF)

ESF is a control function that can automatically detect a potential collision and activate the vehicle's steering system for a limited duration to steer the vehicle in an attempt to avoid or mitigate the collision. ESF can be activated in response to various situations, including when another vehicle in an adjacent lane is drifting toward the subject vehicle or when an obstacle is obstructing the path of the subject vehicle. Here are some key topics mentioned on the regulation regarding ESF systems:

- **Collision Risk Detection:** The ESF should only activate when it detects a risk of a collision.
- **Driving Environment Monitoring:** Vehicles equipped with ESF must have means to monitor the driving environment. This includes monitoring factors like lane markings, the road edge, and other road users while the ESF is active.
- **Avoidance Maneuver Constraints:**
 - ESF-initiated avoidance maneuvers should not lead the vehicle to leave the road.

- If lane markings are present, an ESF-initiated avoidance maneuver should not cause the vehicle to leave the road or cross lane markings, except if it corrects an unintentional drift or a lane change initiated by the drive.
 - If there are no lane markings, a single ESF intervention is permitted, but it should not produce a lateral offset of more than 0.75 meters.
- **Avoidance of Collisions:** ESF interventions should not lead the vehicle to collide with other road users.
- **Monitoring Systems:** Manufacturers must demonstrate to regulatory authorities that the vehicle has systems to monitor the driving environment and fulfill the requirements outlined in the regulation.
- **Driver Feedback:**
 - The ESF intervention must be indicated to the driver with optical (visual), acoustic (auditory), or haptic (tactile) warning signals. These signals should start when the ESF intervention begins.
 - If there is a system failure, an optical warning signal should inform the driver. However, this indication may be suppressed when the system is manually deactivated.
- **Testing:** The vehicle equipped with ESF should undergo specific tests to ensure that it fulfills the requirements. These tests include various scenarios where ESF interventions are tested for effectiveness and safety.

Some of the specific tests regarding the ESF systems are:

1. **Test for ESF Type a i/ii (Unintentional Lateral Maneuver):** In this test, a target vehicle approaches the vehicle under test in the adjacent lane, and one of the vehicles minimizes their lateral separation distance until an ESF intervention is initiated. The test requirements include ensuring that the warnings specified in the regulation are provided before the ESF intervention starts, and the ESF intervention does not lead the vehicle to leave its original lane.

2. **Test for ESF Type a iii (Intentional Lateral Maneuver):** This test simulates an intentional lane change by the vehicle under test while another vehicle is driving in the adjacent lane. The requirements include initiating an ESF intervention, providing specified warnings before the ESF intervention, and ensuring that the ESF intervention does not lead the vehicle to leave its original lane.
3. **Test for ESF Type b (Avoiding an Object):** In this test, the vehicle under test approaches an object positioned within its trajectory. The object should be of a size and position that allows the vehicle to pass it without crossing the lane markings. The requirements involve the ESF avoiding or mitigating the collision, providing warnings before the ESF intervention, and ensuring that the ESF intervention does not lead the vehicle to leave its lane.
4. **Tests for Systems Able to Operate Without Lane Markings:** If the ESF system is designed to operate in the absence of lane markings, the tests described in the previous types need to be repeated on a test track without lane markings. The requirements include ESF intervention initiation, specified warnings, limiting lateral offset during the maneuver, and ensuring that the vehicle does not leave the road due to the ESF intervention.
5. **False Reaction Test for ESF Type b:** In this test, the vehicle under test approaches a plastic sheet placed between lane markings, simulating an object on the road. The test requirements involve ensuring that the ESF does not initiate any intervention.

In summary, I took a deep look into the rules and standards that govern the emergency steering systems in self-driving cars, giving special attention to the Euro NCAP and UN regulations. It's important to note that the rules about emergency steering for autonomous cars are quite recent. This shows that we're just starting to create a worldwide understanding of what's expected and needed for these systems to be safe on our roads. It's clear that there's a real need for more detailed and widespread regulations that can keep up with the quick advances in autonomous vehicle technology.

4. RESEARCH OBJECTIVES AND METHODOLOGY

In this research, two distinct approaches for developing a collision avoidance system within autonomous vehicles will be explored, with a focus on reinforcement learning (RL) strategies, leveraging my knowledge and interest in this area. Both methodologies are implemented and tested within the same simulation environment, utilizing the CARLA Simulator [63], and are based on the same algorithm, Proximal Policy Optimization (PPO). Additionally, they share identical reward functions to ensure consistency in evaluation. The main differences between each approach will be on the action and state space. The objective is to compare and analyze the advantages and disadvantages of each approach following their development and testing phases.

The first approach employs a hybrid approach (Chapter 5), combining Reinforcement Learning (RL) with a Model Predictive Control (MPC) controller. The RL component utilizes the Proximal Policy Optimization (PPO) algorithm [77], while the MPC is developed using Python's cvxpy library [78]. The proposed framework is a combination between a Proximal Policy Optimization (PPO) agent, a trajectory planner, and a Model Predictive Control (MPC) controller, all operating within the CARLA simulator environment.

The second approach (Chapter 6) presents an alternative approach to the Model Predictive Control (MPC) method. Unlike the MPC, which relies on a model of the vehicle's dynamics and environmental constraints, this chapter explores an end-to-end control strategy. This approach directly maps raw sensory inputs to control actions through deep learning. Specifically, images from a camera was used as the input state space, representing the vehicle's immediate environment. The action space consists of the vehicle's steering and acceleration controls, making the system capable of learning complex driving policies directly from visual inputs.

4.1. CARLA SIMULATION ENVIRONMENT

Carla Simulator has a four-wheel model that can be used for both training and evaluation. CARLA primarily uses a physics engine for vehicle dynamics simulation. The simulator is integrated with the Unreal Engine 4 (UE4), leveraging UE4's built-in physics

engine, which is based on PhysX by NVIDIA [79]. The NVIDIA PhysX vehicle model simulates vehicles using rigid body actors to represent chassis and suspensions. It calculates forces from tires and suspensions, applying them to simulate motion and handling. The model supports various drivetrain configurations, reflecting engine torque on wheels through clutches and differentials for accurate vehicle behavior. More detailed information can be seen on NVIDIA documentation [79].

4.2. TRAINING SCENARIO

The training scenario used in our simulation is shown in Figure 10. This setup includes an ego car, managed by the RL agent, and a stationary car positioned at a suitable distance. This arrangement allows the ego car to accelerate to the target speed. The scenario was designed to mirror the Car-to-Car Rear Stationary (CCRs) test from Euro NCAP. For training, I selected a Tesla Model 3 from the Carla simulator as the ego car. Initially, the ego car follows a basic motion planning algorithm, moving straight until it achieves a desired speed.

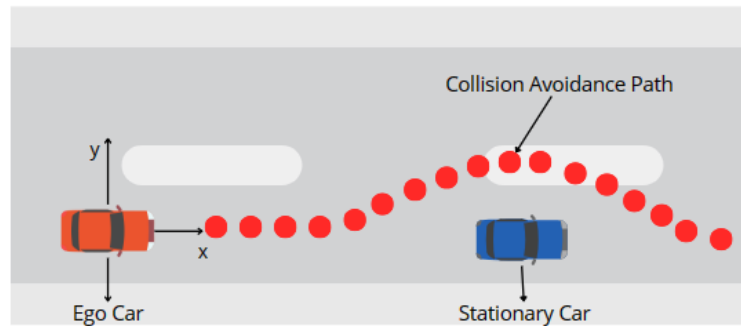


Figure 10: Simulation Scenario

After the desired speed is reached, a Time-to-Collision (TTC) metric is employed, expressed by equation 4.1, where X_s , V_s and X_e , V_e are the position and velocity of the stationary car and ego car, respectively, where in this case $V_s = 0$. To activate the collision avoidance system, the system is set to trigger at a TTC of 1.0 seconds. This value was chosen based on trials, considering scenarios where braking alone is insufficient to avoid a collision, necessitating steering intervention.

$$TTC = \frac{X_s - X_e}{V_e - V_s} \quad (4.1)$$

Once the system is triggered, the PPO agent is responsible for providing the waypoints and desired speed at each timestep.

4.3. PROXIMAL POLICY OPTIMIZATION (PPO)

Proximal Policy Optimization (PPO) is a popular reinforcement learning algorithm that addresses some of the limitations of earlier policy optimization methods. It was introduced by Schulman et al. in 2017 [77], and has become widely adopted due to its stability, efficiency, and ease of implementation. Some key concepts:

1. Objective Function:

- PPO aims to optimize the policy by maximizing the expected cumulative reward, similar to other policy gradient methods and uses a stochastic policy, as opposed to the deterministic one [80].

2. Clipping Mechanism:

- To prevent large policy updates that could destabilize training, PPO introduces a clipping mechanism.
- The objective is modified to penalize excessively large policy changes, ensuring that the new policy does not deviate too far from the old policy in a single update [77].

3. Surrogate Objective:

- The surrogate objective in PPO is defined to ensure stable policy updates while maximizing expected rewards.
- It includes a minimum between two terms: the clipped surrogate and an entropy term that encourages exploration [81].

In summary, the PPO objective function aims to maximize a clipped surrogate objective function. It is designed to prevent the new policy from straying too far from the old policy, thereby avoiding large and potentially harmful policy updates. The objective function [77] can be described by equation 4.2, as follows:

$$L^{CLIP}(\theta) = \hat{E}_t [\min (r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t)] \quad (4.2)$$

Here's what each term represents:

- $L^{CLIP}(\theta)$: The clipped surrogate objective function. This is what PPO aims to maximize.
- θ : The parameters of the policy neural network.
- $\hat{E}_t$: The expected value over a finite batch of samples.
- $r_t(\theta)$: The probability ratio, this ratio measures how the new policy diverges from the old policy.
- $\hat{A}_t$: The advantage estimate at time t, which measures the benefit of taking an action compared to the expected value.
- $\text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)$: This operation clips the probability ratio within the interval $[1 - \epsilon, 1 + \epsilon]$, with ϵ being a small positive value (like 0.2). It's used to define the limits within which the new policy can change, compared to the old policy.
- $\min(\cdot, \cdot)$: The minimum between the unclipped and clipped objectives. This ensures that the objective does not benefit from changes that move outside of the interval $[1 - \epsilon, 1 + \epsilon]$.

The rationale behind the clipped objective is to take the smaller value between the new and old policies, which prevents the policy updates from being too large and causing instability in the learning process. This clipping mechanism is a key component that differentiates PPO from other policy gradient methods and has been instrumental in its success across a variety of reinforcement learning tasks.

Considering the stability, efficiency, and ease of implementation. As well as the fact that the PPO algorithm can handle continuous space it makes this algorithm a good candidate for a collision avoidance system.

4.4. REWARD FUNCTION

The reward function is the signal that the agent uses to learn the optimal policy. It assesses the desirability of an agent's actions based on the current state and action taken. The objective of the agent is to maximize the cumulative reward. For this collision avoidance system the reward function was inspired by [82] and adapted to the collision avoidance problem, both approaches have the same reward functions and the reward function is defined as:

$$Reward = \lambda_1 \cos \theta + \lambda_2 |d| + \lambda_3 C - \lambda_4 L + \lambda_5 T \quad (4.3)$$

Where:

- $\cos \theta$: The cosine of the difference between the yaw of the ego vehicle and the yaw corresponding to the correct orientation on the road, represented by Figure 11. This term encourages the vehicle to align with the road. Even though we want the vehicle to avoid the obstacle by steering. This term is important, so the vehicle maintains its stability.

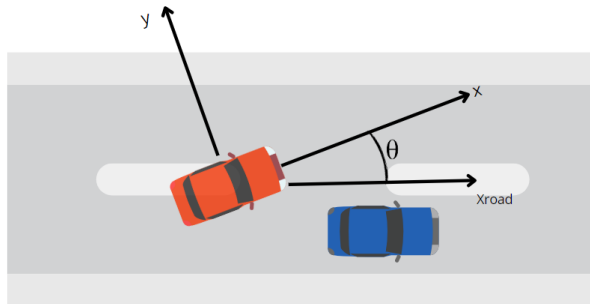


Figure 11: The cosine of the difference between the yaw of the ego vehicle and the yaw corresponding to the correct orientation on the road.

- d : The distance of the ego vehicle from the optimal path or the lane center. This term penalizes the agent for deviating from the path.

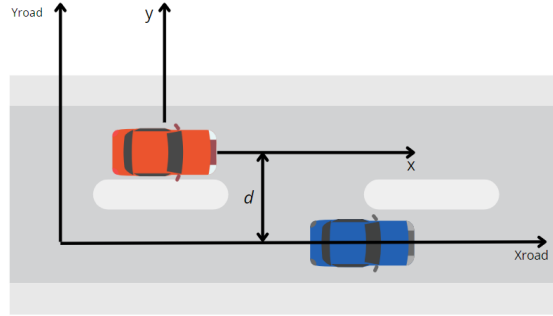


Figure 12: The distance of the ego vehicle from the optimal path or the lane center.

- C (collision indicator): A binary value that indicates whether a collision has occurred. This term heavily penalizes collisions to ensure safety is the top priority.

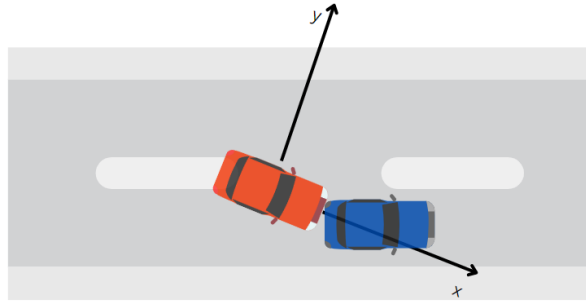


Figure 13: Collision Indicator

- L (lane invasion indicator): A binary value that indicates whether the ego vehicle is within the correct lane boundaries. This term penalizes the crossing of solid lane markings.

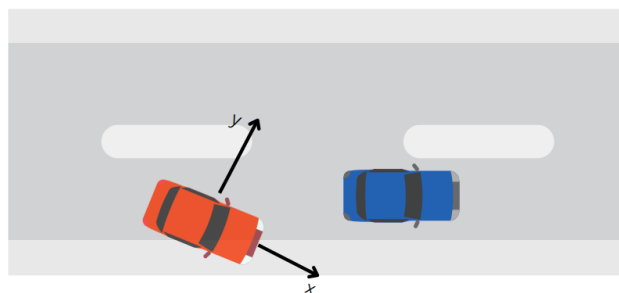


Figure 14: Lane Invasion Indicator

- T : The distance traveled in the correct direction. This term rewards forward progress along the route.

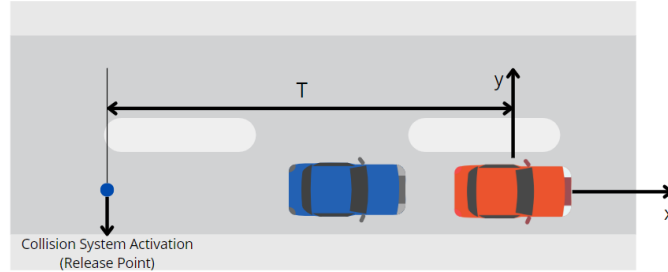


Figure 15: The distance traveled.

The weights ($\lambda_1, \lambda_2, \lambda_3, \lambda_4, \lambda_5$) are hyperparameters that can be adjusted based on the importance of each term in the context of the driving task. By fine-tuning these weights, the system can be calibrated to prioritize different aspects of the driving behavior. For the collision avoidance problem, after fine-tuning, the set hyperparameters are:

$$\lambda_1 = 1, \quad \lambda_2 = 1, \quad \lambda_3 = 100, \quad \lambda_4 = 5, \quad \lambda_5 = 0.5$$

An episode is terminated under any of the following conditions:

- A collision occurs.
- The ego vehicle crosses a solid lane marking.
- The lateral distance of the ego car from the right lane exceeds 4.5 meters.
- The vehicle stops completely. Here a penalty of -50 was added to motivate the model to drive past the car and not fully stop.
- The vehicle arrives at a predetermined goal position, which is set 15 meters in front of a stationary car.

Both approaches will be detailed on the following chapters.

5. DESIGN AND IMPLEMENTATION OF A HYBRID COLLISION AVOIDANCE SYSTEM USING REINFORCEMENT LEARNING AND MODEL PREDICTIVE CONTROL

In this chapter, the methodology used to develop the hybrid collision avoidance system for autonomous vehicles is detailed. This system employs a hybrid approach, combining Reinforcement Learning (RL) with a Model Predictive Control (MPC) controller. The RL component utilizes the Proximal Policy Optimization (PPO) algorithm [77], while the MPC is developed using Python's cvxpy library [78]. The training and testing are conducted in the CARLA simulator [63]. The full code implementation can be seen on my GitHub page¹.

5.1. PROPOSED FRAMEWORK

The proposed framework is depicted in Figure 16. The collision avoidance system presented in this approach is a combination between a Proximal Policy Optimization (PPO) agent, a trajectory planner, and a Model Predictive Control (MPC) controller, all operating within the CARLA simulator environment.

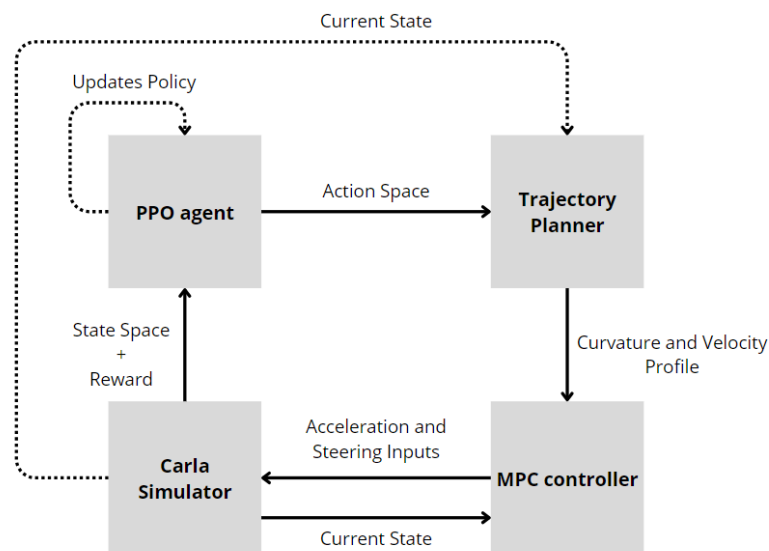


Figure 16: Proposed Hybrid Collision Avoidance Framework

¹ [gustavomoers/CollisionAvoidance-Carla-DRL-MPC. \(github.com\)](https://github.com/gustavomoers/CollisionAvoidance-Carla-DRL-MPC)

The framework initiates with the PPO agent, a reinforcement learning module responsible for making decisions based on the current state of the vehicle and the environment. It generates an action space that represents the ideal trajectory and speed required to avoid collisions.

Subsequently, the action space serves as input for the trajectory planner. This component computes a curvature and velocity profile, which translates the desired trajectory into a smoother path composed of waypoints. These waypoints guide the vehicle's movement, ensuring that the transition from one point to another is both smooth and feasible within the vehicle's operational constraints.

The curvature and velocity profile are then fed into the MPC controller. Using the kinematic bicycle model, the MPC controller calculates the necessary acceleration and steering inputs to follow the planned path while considering the vehicle dynamics and physical limits. The control commands generated by the MPC are precise and adaptive, taking into account the current state of the vehicle and the immediate requirements of the path and speed profile.

The MPC's commands are fed back into the CARLA simulator. This feedback loop provides the PPO agent with updated state information and reward assessments to improve its policy, leading to a continuous learning and cycle.

In the following sections, each component of the framework will be explored in detail, explaining their individual contributions to the overall functionality of the collision avoidance system.

5.2.SCOPE AND LIMITATIONS

The system integrates Reinforcement Learning (RL) with Model Predictive Control (MPC) to navigate and avoid potential collisions in a simulated environment. Key components of the methodology include:

- Utilization of the CARLA simulator for creating a realistic urban driving environment.
- Implementation of the Proximal Policy Optimization (PPO) algorithm for training the RL agent.

- Development of a detailed action and state space to guide the RL agent.
- Formulation of a complex reward function to optimize the agent's performance.
- Application of a kinematic bicycle model to represent vehicle dynamics.
- Design of a path planner using cubic spline curves.
- Creation of an MPC controller, using Python's cvxpy library, to manage real-time vehicle control.

While the methodology presented in this chapter offers a robust framework for developing a collision avoidance system, there are limitations that must be acknowledged:

- **Simulation Constraints:** The CARLA simulator may not capture all the nuances of real-world driving conditions. Consequently, the system's performance in the simulator might differ from that in actual environments.
- **Model Simplifications:** The kinematic bicycle model, despite its effectiveness in capturing vehicle dynamics, is a simplification. Real-world scenarios may require more complex models to account for factors like tire dynamics and nonlinear vehicle responses.
- **Computational Demands:** The RL training process, particularly with PPO, is computationally intensive and requires significant resources for optimal performance.
- **Generalizability:** The training scenarios, focused on a specific test setup, might limit the system's adaptability to diverse or unexpected situations.
- **Control Constraints:** The MPC controller's effectiveness is subject to the accuracy of the model predictions and the computational efficiency of the optimization process.

These limitations are important considerations in the interpretation of the system's capabilities and its potential application in real-world scenarios.

5.3. STATE SPACE

The state space encapsulates all the possible configurations in which the agent can exist. For the autonomous driving system developed in this approach, the state space is a

representation of all the pertinent information that the vehicle needs to make informed decisions about its actions. Specifically, the state space is represented by 4.3, which are described by Figure 17:

$$\text{State Space} = [(X_{stat} - X_{ego}), (Y_{stat} - Y_{ego}), V_{ego}, \dot{V}_{ego}, \varphi_{ego}] + \text{Previous Actions} \quad (5.1)$$

Where each term is described as:

- $(X_{stat} - X_{ego})$: The longitudinal distance from the ego vehicle to the stationary car ahead.
- $(Y_{stat} - Y_{ego})$: The lateral distance from the ego vehicle to the centerline of the lane or to a reference path.
- V_{ego} : The current velocity of the ego vehicle.
- $\dot{V}_{ego}$: The rate at which the ego vehicle's speed is changing.
- φ_{ego} : The orientation of the ego vehicle with respect to the stationary car or the road.
- *Previous Actions*: An array storing the previous actions taken by the vehicle, which can be crucial for understanding the vehicle's current dynamic state and for smoother paths.

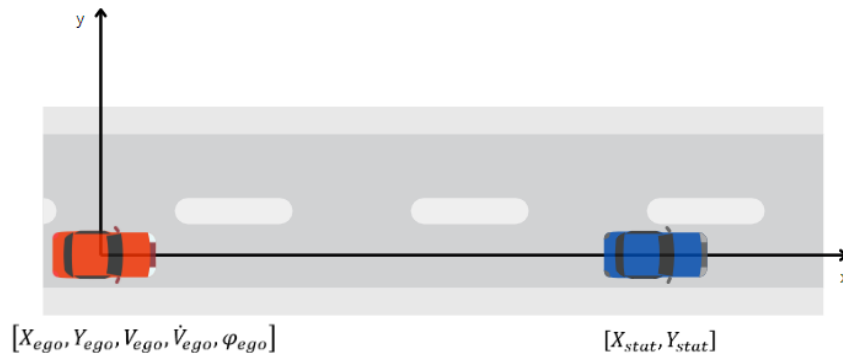


Figure 17: State Space

The state variables are normalized to ensure uniformity in scale, which helps in the learning process by maintaining numerical stability and often leads to faster convergence.

5.4. ACTION SPACE

The action space defines the set of all possible actions that the agent can take at any given state. In the context of this approach, the action space is continuous and is represented by 4.4 and Figure 18:

$$\text{Action Space} = [x_1, y_1, x_2, y_2, v_{desired}] \quad (5.2)$$

Where:

- **Waypoints:** The (x_1, y_1, x_2, y_2) coordinates representing the trajectory that the vehicle should follow. These waypoints are collision-free and dynamically calculated based on the vehicle's current state and the environment. These waypoints are later fitted on a cubic spline curve to guarantee the smooth and feasible path.
- **Desired Speed ($v_{desired}$):** The target velocity that the vehicle should attempt to reach or maintain. This is a critical component, as the agent must learn to adjust the speed appropriately to navigate safely and efficiently. The desired speed is then linear interpolated between the current speed and the desired one over the horizon plan.

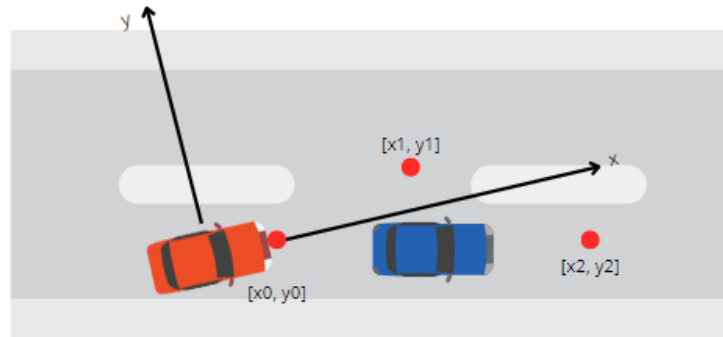


Figure 18: Action Space

5.5. TRAJECTORY PLANNER

Trajectory planning is a step that involves determining a viable path and corresponding velocity profile for the vehicle to follow. For this collision avoidance system, the trajectory planning module receives input from the PPO agent in the form of desired waypoints and speeds. It translates these inputs into a precise trajectory that the vehicle should follow, which includes both the path the vehicle should take (spatial component) and the speed at which it should travel (temporal component), both are generated at each timestep and the path is continuously replanned.

5.5.1 CUBIC SPLINE CURVE

To ensure that the trajectory between waypoints is smooth and drivable, a cubic spline curve is used. A spline is a piecewise polynomial function that can provide a smooth and continuous curve between a set of points. In the case of a cubic spline, each piece of the curve is defined by a cubic polynomial, which means that the curve has continuous first and second derivatives, resulting in smooth transitions between waypoints [83].

The cubic spline curve for the path planning is mathematically formulated as follows:

Given a set of waypoints (x_i, y_i) , where i indexes the waypoints, the cubic spline curve $S(x)$ is a function such that:

- $S(x_i) = y_i$, which means that the curve passes through all waypoints.
- The first and second derivatives of $S(x)$ are continuous across the different segments of the spline.
- The second derivative of $S(x)$ at the endpoints (first and last waypoints) is set to zero (natural spline boundary conditions)

The cubic spline generates a path that is not only smooth and easy to follow but also computationally efficient to calculate. Additionally, by adjusting the waypoints, the vehicle can dynamically change its path in response to the environment, such as when avoiding an obstacle or a suddenly stopped car.

To generate the cubic spline curve for this system 3 waypoints were given, the current state (x_0, y_0) and the learned waypoints (x_1, y_1, x_2, y_2) , the visual representation can be seen

on Figure 19. From these 3 waypoints the cubic spline curve was generated using the module Cubic Spline Planning from the python package PythonRobotics [83], more details about the implementation can be seen on the documentation. The output from the planner consists of a sequence of waypoints $[x_t, y_t, \varphi_t, v_t]$ spaced one meter apart, alongside the desired speed at each waypoint.

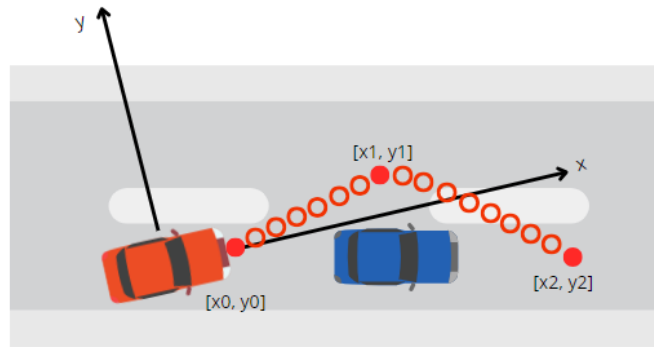


Figure 19: Cubic Spline Curve

5.5.2 VELOCITY PROFILE GENERATION

Once a path has been determined by the path planner using a cubic spline curve, the next step is to generate a velocity profile. A velocity profile dictates the speed at which the vehicle should travel along the path.

In this scenario, the velocity profile is determined by linear interpolating between the current speed and the desired speed (given by the action space) along the horizon plan from the MPC controller.

5.6. MODEL PREDICTIVE CONTROLLER

The path and velocity profile together form a comprehensive plan for the vehicle's movement. This plan is then utilized by the MPC controller, which computes the control inputs needed to follow the trajectory as closely as possible. The MPC considers the vehicle's current state, and the planned trajectory to determine the optimal steering angles, throttle, and brake commands.

In this project, the MPC was adapted from [84] and modified to our environment and problem. The Linear MPC controller is implemented using CVXPY, a Python-embedded modeling language for convex optimization problems. CVXPY allows for the straightforward formulation of the MPC problem and uses solvers like ECOS [78].

According to Kong et al. [85], the kinematic model demonstrates good performance comparable to a dynamic model. Therefore, the proposed approach uses the kinematic model to achieve reduced computational power. The prediction horizon, N , was set to 8 and the discretization timestep, dt , was set to 0.2. Both were optimized through fine-tuning to balance accuracy with computational efficiency.

5.6.1 OBJECTIVE FUNCTION

The objective function aims to minimize the deviation from the reference trajectory and control effort, including penalties for rapid changes in control inputs. It can be represented as:

$$\min_u \sum_{t=0}^{N-1} \left(\|z_{ref,t} - z_t\|_Q^2 + \|u_t - u_{t-1}\|_R^2 + \|u_t\|_{\bar{R}}^2 \right) + \|z_{ref,N} - z_N\|_{Q_f}^2 \quad (5.3)$$

Where:

- N is the prediction horizon length.
- $z_{ref,t}$ and z_t are the reference and actual state vectors at timestep t respectively. The state vector is composed of $[x, y, v, \varphi]$.
- u_t is the control input vector at timestep t composed of $[a, \delta]$, acceleration and steering angle, respectively.
- Q , R and $\bar{R}$ are diagonal matrices comprising the cost weights.
- Q_f is the terminal state weighting matrix.
- The notation $\|x\|_A^2$ denotes the quadratic form $x^T A x$.

The code that defines the linear MPC cost function can be seen on 11.1, and the full MPC implementation and connection with CARLA and the RL model can be seen on 1.

5.6.2 SYSTEM DYNAMICS AND LINEARIZATION

The system dynamics are approximated by the linear model:

$$x_{t+1} = Ax_t + Bu_t + C \quad (5.4)$$

Where:

- x_t is the state vector at timestep i .
- u_t is the control input vector at timestep t
- A , B and C are matrices representing the linearized dynamics.

The linearization process involves calculating the Jacobian matrices of the system dynamics with respect to both the state variables and control inputs.

In MATLAB, the symbolic math toolbox can be used to derive A , B and C matrices. The MATLAB code for the linearization can be seen on 11.2. Here is an example of how this linearization is performed:

$$f = [\dot{x} \ d_t, \dot{y} \ d_t, \dot{v} \ d_t, \dot{\phi} \ d_t] \quad (5.5)$$

Where, the time equations that describe a kinematic bicycle model are represented by the following equations and Figure 20 [85]:

$$\dot{x} = v \cos(\varphi + \beta) \quad (5.6)$$

$$\dot{y} = v \sin(\varphi + \beta) \quad (5.7)$$

$$\dot{v} = a \quad (5.8)$$

$$\dot{\phi} = \frac{v}{l_r} \sin(\beta) \quad (5.9)$$

$$\beta = \tan^{-1} \left(\frac{l_r}{l_f + l_r} \tan \delta \right) \quad (5.10)$$

Where x and y are the coordinates of the center of mass in an inertial frame (X, Y) . d_t term represents the discretization timestep for the MPC. φ is the inertial heading and v is the speed of the vehicle. l_f and l_r represent the distance from the center of the mass of the vehicle to the front and rear axles, respectively. β is the angle of the current velocity of the center of mass with respect to the longitudinal axis of the car. a is the acceleration of the center of mass in the same direction as the velocity. The control inputs are the steering angle δ , and a .

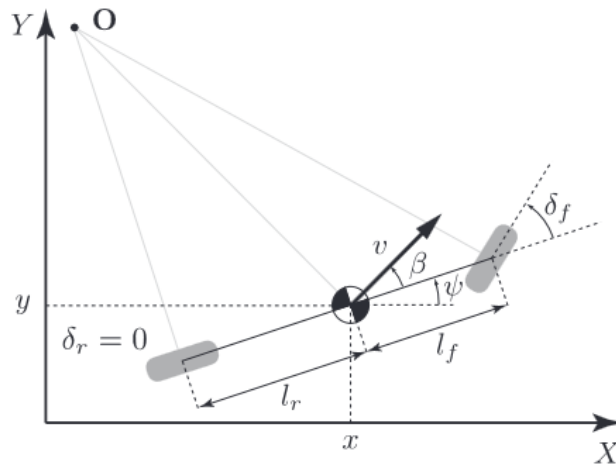


Figure 20: Kinematic Bicycle Model [85]

Once the equations are defined, the linearization can be done by the following equations on MATLAB:

$$A = \text{jacobian}(f, [x, y, v, \varphi]) \quad (5.11)$$

$$B = \text{jacobian}(f, [a, \delta]) \quad (5.12)$$

$$X = [x, y, v, \varphi] \quad (5.13)$$

$$U = [a, \delta] \quad (5.14)$$

$$C = f - AX - BU \quad (5.15)$$

From MATLAB, the derived matrices are:

$$A = \begin{bmatrix} 1 & 0 & \cos(\varphi + \beta) & -v \cdot \sin(\varphi + \beta) \\ 0 & 1 & \sin(\varphi + \beta) & v \cdot \cos(\varphi + \beta) \\ 0 & 0 & 1 & 0 \\ 0 & 0 & \frac{\tan(\delta)}{L\sqrt{1+\frac{l_r^2 \cdot \tan^2(\delta)}{L^2}}} & 1 \end{bmatrix} dt \quad (5.16)$$

$$B = \begin{bmatrix} 0 & \frac{-l_r \cdot v \cdot (\tan^2(\delta) + 1) \cdot \sin(\varphi + \beta)}{L \cdot (1 + \frac{l_r^2 \cdot \tan^2(\delta)}{L^2})} \\ 0 & \frac{l_r \cdot v \cdot (\tan^2(\delta) + 1) \cdot \cos(\varphi + \beta)}{L \cdot (1 + \frac{l_r^2 \cdot \tan^2(\delta)}{L^2})} \\ 1 & 0 \\ 0 & \left(\frac{v \cdot (\tan^2(\delta) + 1)}{L\sqrt{1+\frac{l_r^2 \cdot \tan^2(\delta)}{L^2}}} - \frac{l_r^2 \cdot v \cdot (\tan^2(\delta) + 1) \cdot \tan^2(\delta)}{L^3 \cdot \sqrt[3]{1+\frac{l_r^2 \cdot \tan^2(\delta)}{L^2}}} \right) \end{bmatrix} dt \quad (5.17)$$

$$C = \begin{bmatrix} v \cdot \varphi \cdot \sin(\varphi + \beta) + \frac{\delta \cdot l_r \cdot v \cdot (\tan^2(\delta) + 1) \cdot \sin(\varphi + \beta)}{L \cdot (1 + \frac{l_r^2 \cdot \tan^2(\delta)}{L^2})} \\ -v \cdot \varphi \cdot \cos(\varphi + \beta) - \frac{\delta \cdot l_r \cdot v \cdot (\tan^2(\delta) + 1) \cdot \cos(\varphi + \beta)}{L \cdot (1 + \frac{l_r^2 \cdot \tan^2(\delta)}{L^2})} \\ 0 \\ -\delta \cdot \frac{v \cdot (\tan^2(\delta) + 1)}{L\sqrt{1+\frac{l_r^2 \cdot \tan^2(\delta)}{L^2}}} - \frac{l_r^2 \cdot v \cdot (\tan^2(\delta) + 1) \cdot \tan^2(\delta)}{L^3 \cdot \sqrt[3]{1+\frac{l_r^2 \cdot \tan^2(\delta)}{L^2}}} \end{bmatrix} dt \quad (5.18)$$

5.6.3 CONSTRAINTS

The MPC optimization problem incorporates several constraints to ensure realistic behavior:

1. **Steering Constraints:** Limits the steering angle to the vehicle's physical capabilities.

$$-\delta_{max} \leq \delta \leq \delta_{max} \quad (5.19)$$

2. **Acceleration Constraints:** Ensures the acceleration is within a feasible range.

$$a_{min} \leq a \leq a_{max} \quad (5.20)$$

3. **Velocity Constraints:** Keeps the vehicle's speed within limits.

$$v_{min} \leq v \leq v_{max} \quad (5.21)$$

4. **Rate of Change Constraints:** Controls the rate of change of acceleration and steering to avoid abrupt movements.

$$-\dot{a}_{max} \leq \dot{a} \leq \dot{a}_{max} \quad (5.22)$$

$$-\dot{\delta}_{max} \leq \dot{\delta} \leq \dot{\delta}_{max} \quad (5.23)$$

Where, for this scenario the defined parameters are:

- $\delta_{max} = 0.61 \text{ rad}$, which corresponds to 35° .
- $a_{min} = -1.5$ and $a_{max} = 2 \frac{m}{s^2}$
- $v_{min} = 0$ and $v_{max} = 40 \text{ m/s}$
- $\dot{a}_{max} = 1.5 \frac{m}{s^3}$
- $\dot{\delta}_{max} = 0.3 \text{ rad/s}$

5.7. NETWORK ARCHITECTURE AND HYPERPARAMETERS

To implement the reinforcement learning algorithm, the Stable Baselines3 framework [86] was used, which offers a collection of robust reinforcement learning algorithms built on PyTorch. For this model the recurrent version of PPO (Recurrent PPO) [87] was used, integrating LSTM for better handling of sequential data.

Initial trials with the default PPO configuration revealed challenges in achieving convergence, with the model frequently getting stuck in local optima. The shift to Recurrent PPO, incorporating LSTM layers, improved the model's ability to navigate the simulation scenario by effectively utilizing past and current data for decision-making.

Initially, the default hyperparameters were tested to assess the model's convergence. After some fine-tuning, most hyperparameters were maintained as initially set, except for the entropy coefficient, which was adjusted to 0.01. This adjustment proved important for improving the agent's exploration capabilities, preventing the model from converging to suboptimal local minima.

Table 2: Network Hyperparameters Hybrid Approach

Hyperparameter	Value
Learning Rate	0.0003
Number of Steps	640
Batch Size	32
Number of Epochs	30
Discount Factor (γ)	0.99
Entropy Coefficient	0.01

The architecture of the neural network was maintained as per the framework's default configuration. The network features a Feature Extractor for pre-processing input data, coupled with LSTM layers for both the policy (actor) and value (critic) networks. This structure enables the model to maintain a memory of past states. Subsequent to feature extraction, two fully connected networks map these features to the actions and value estimates, respectively. The action network specifically outputs 5 values, corresponding to the defined action space.

Table 3: Network Architecture

Component	Layer	Configuration
Feature Extractor	Policy Net	Linear(256, 64) > Tanh > Linear(64, 64) > Tanh
	Value Net	Linear(256, 64) > Tanh > Linear(64, 64) > Tanh
LSTM Layer	LSTM Actor	LSTM(10, 256)
	LSTM Critic	LSTM(10, 256)
Action Network	Linear	Linear(64, 5)
Value Network	Linear	Linear(64, 1)

5.8. TRAINING RESULTS

The training was conducted on, an i7-10510U PC with an NVIDIA GeForce MX110, taking approximately 44 hours for 143,500 steps. A curriculum learning strategy was employed, progressively training the model at 50km/h, 70km/h, and finally 90km/h. The rewards per step over these phases are depicted in Figure 21, illustrating the model's learning evolution.

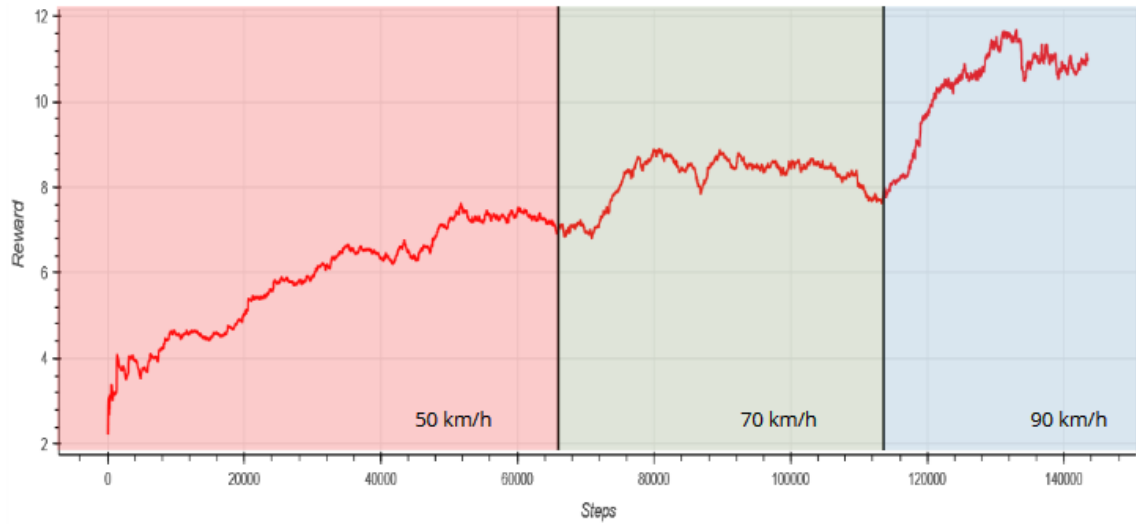


Figure 21: Reward per Step: Hybrid Approach

Ultimately, a single model was developed which is capable of managing collision avoidance for speeds ranging from 50 to 90 km/h within the specific training scenario. Graphical results showing this are presented below, demonstrating the model's effectiveness across the tested speed intervals.

5.9. TESTING

The learned model was evaluated over 30 episodes for each speed. Figure 22, Figure 23, and Figure 24 showcase the ego car's trajectory at these speeds, illustrating the model's consistent path-following. This consistency is attributed to a highly descriptive state space that remains uniform across different scenarios. The testing and the results can be seen on this youtube video².

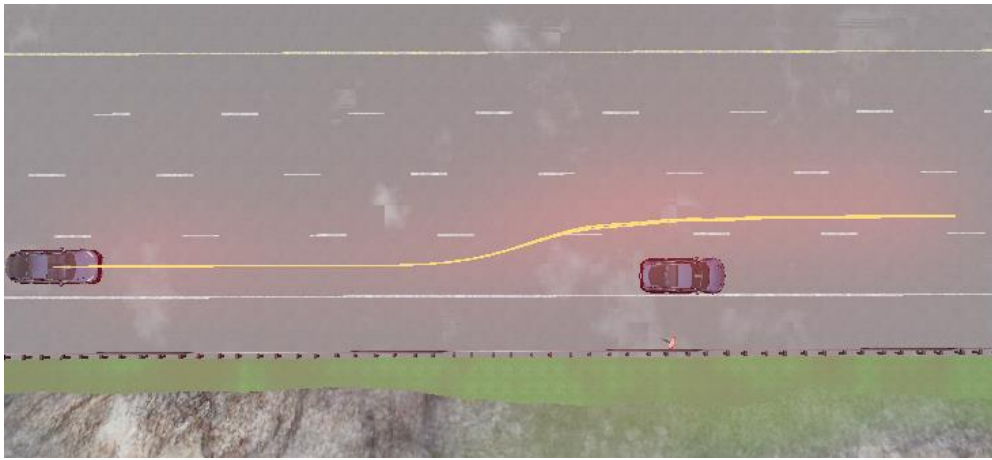


Figure 22: Path followed by ego car at 50 km/h using the hybrid approach agent.



Figure 23: Path followed by ego car at 70 km/h using the hybrid approach agent.

² [Collision Avoidance - Hybrid Approach - Reinforcement Learning + MPC: results](#)

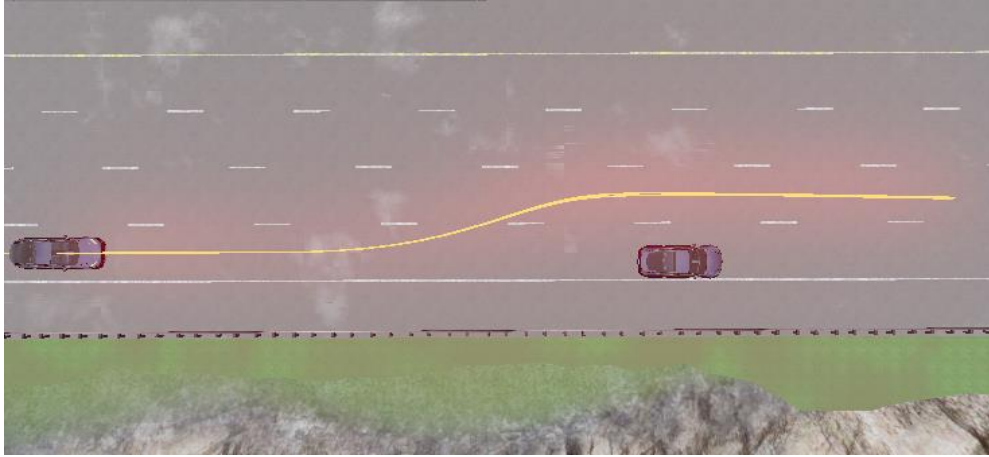


Figure 24: Path followed by ego car at 90 km/h using the hybrid approach agent.

The ego car's path, is shown in Figure 25, illustrating effective collision avoidance via steering. Notably, the system is activated earlier at higher speeds due to a Time-to-Collision (TTC) metric. In this model, the car remains in its lane post-maneuver and does not return to its original lane.

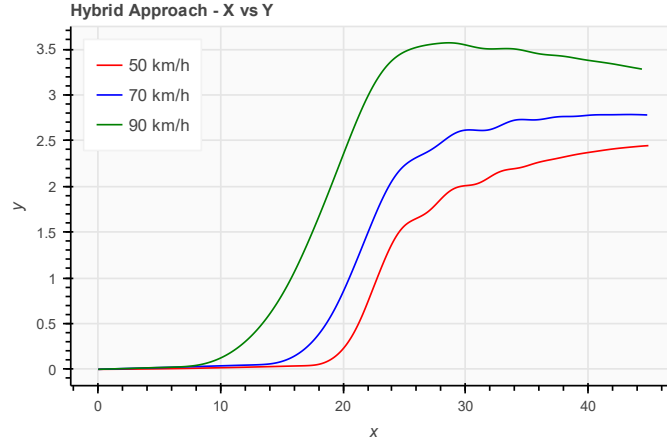


Figure 25: Path taken by the agent at 50, 70 and 90 km/h: Hybrid Approach

Figure 26 displays the speed profile, where the car decelerates to 20km/h across all scenarios, this could be associated to the reward function that favours distance traveled over full braking.

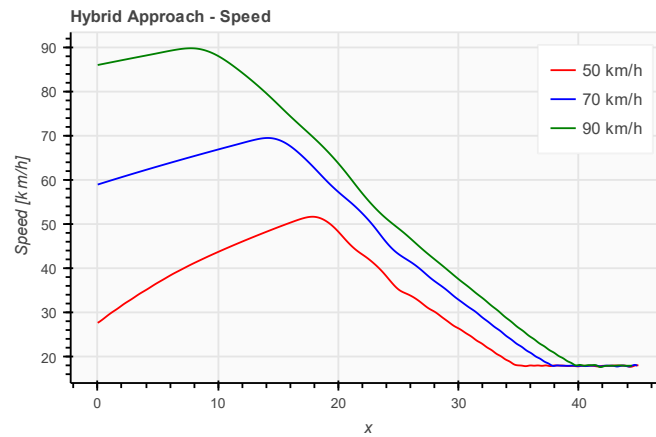


Figure 26: Speed: Hybrid Approach

Figure 27 and Figure 28 illustrate the model's smooth steering transitions and lateral accelerations, achieved through the integration of an MPC that accounts for such dynamics.

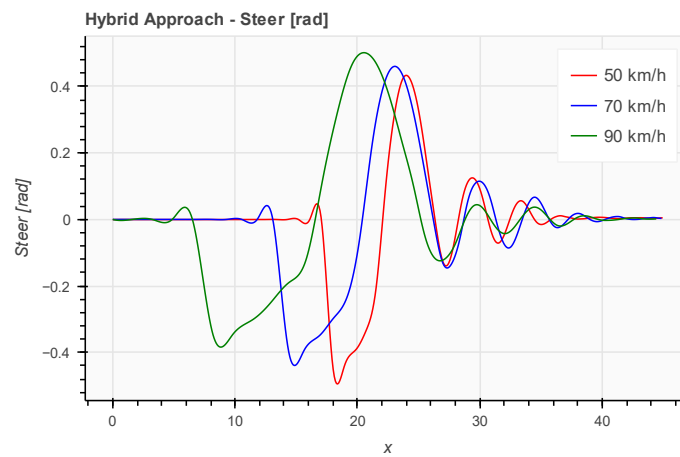


Figure 27: Steer: Hybrid Approach.

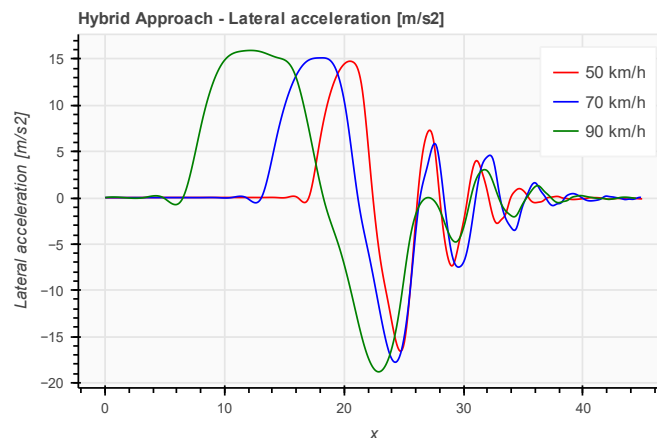


Figure 28: Lateral Acceleration: Hybrid Approach.

6. DESIGN AND IMPLEMENTATION OF AN END-TO-END APPROACH FOR A COLLISION AVOIDANCE SYSTEM

This chapter presents an alternative approach to the Model Predictive Control (MPC) method discussed in the previous chapter. Unlike the MPC, which relies on a model of the vehicle's dynamics and environmental constraints, this method explores an end-to-end control strategy. This approach directly maps raw sensory inputs to control actions through deep learning. Specifically, images from a camera will be used as the input state space, representing the vehicle's immediate environment. The action space consists of the vehicle's steering and acceleration controls, making the system capable of learning driving policies directly from visual inputs. The full code implementation can be seen on my GitHub page³.

6.1. PROPOSED FRAMEWORK

The proposed framework is depicted in Figure 29. The collision avoidance system presented in this approach is an end-to-end approach of an PPO agent operated within the CARLA simulator environment.

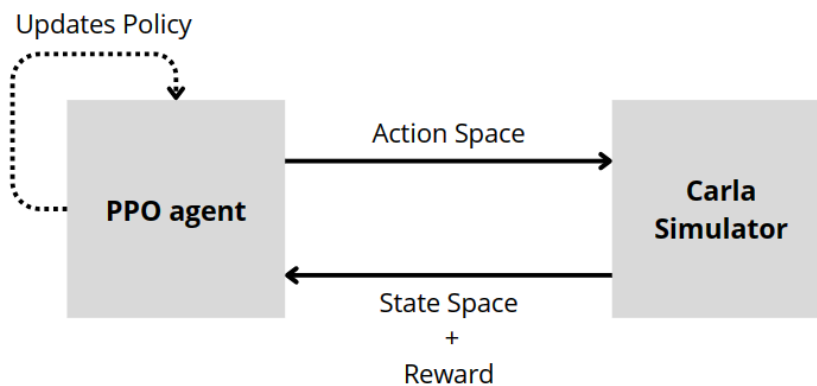


Figure 29: Proposed End-to-End Collision Avoidance Framework

³ [gustavomoers/E2E-CARLA-ReinforcementLearning-PPO. \(github.com\)](https://github.com/gustavomoers/E2E-CARLA-ReinforcementLearning-PPO)

The framework begins with capturing real-time visual data from the vehicle's surroundings, which is then resized and converted to grayscale to standardize input while reducing the model's complexity. This preprocessed visual input serves as the state space for the PPO agent, which is trained to output directly control actions—acceleration and steering angles. This direct mapping from visual observations to control commands offers a more flexible and intuitive approach to autonomous navigation.

6.2.SCOPE AND LIMITATIONS

The end-to-end approach aims to simplify the decision-making process by directly mapping environmental visuals to control actions. While the methodology presented in this chapter offers a framework potentially capable of adapting to a wide range of driving conditions with sufficient training data., there are limitations that must be acknowledged:

Limitations:

- **Environmental Variability:** Performance may vary under different lighting or weather conditions due to the use of visual inputs.
- **Computational Requirements:** Real-time image processing and decision-making demand significant computational resources.
- **Interpretability Challenges:** The "black box" nature of reinforcement learning makes understanding and diagnosing decision-making processes difficult.
- **Simulation Constraints:** The CARLA simulator may not capture all the nuances of real-world driving conditions. Consequently, the system's performance in the simulator might differ from that in actual environments.
- **Generalizability:** The training scenarios, focused on a specific test setup, might limit the system's adaptability to diverse or unexpected situations.

These limitations are important considerations in the interpretation of the system's capabilities and its potential application in real-world scenarios.

6.3. STATE SPACE

The ego vehicle is equipped with a RGB camera sensor. The RGB camera acts as a regular camera capturing images from the scene. The camera is installed in front of the car, above the car's hood and is configured with some parameters: `image_width = 640`, `image_height = 480` and `field_of_view = 110`.

To improve training and reduce the number of parameters of the model the image is preprocessed where it is converted to grayscale, and it is resized. This is done using python package OpenCV. The pre-processing can be seen on Figure 30.

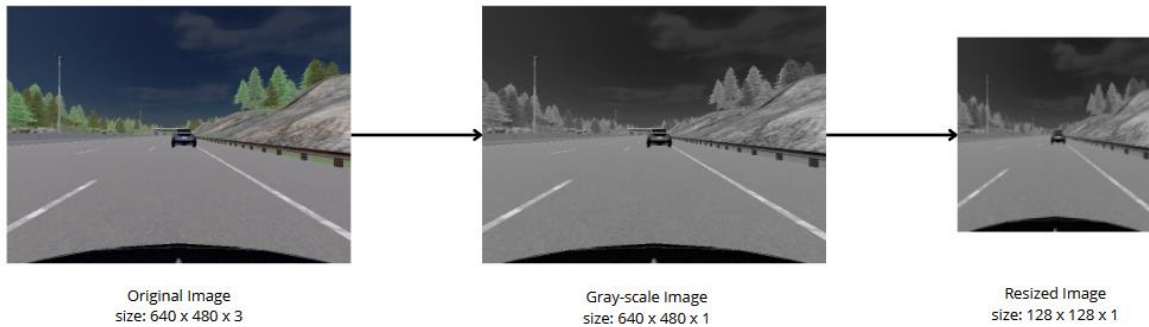


Figure 30: RGB camera pre-processing.

After the pre-processing the image matrix with shape (128, 128, 1) is given to the reinforcement learning algorithm.

6.4. ACTION SPACE

In this end-to-end approach, the action space consists of two continuous variables: acceleration and steering. Acceleration values range from -1 to 1, where -1 represents full braking and 1 means full acceleration. Similarly, steering values also range from -1 to 1, with -1 indicating a full left turn (equivalent to -35 degrees) and 1 denoting a full right turn (35 degrees).

6.5.NETWORK ARCHITECTURE AND HYPERPARAMETERS

In this end-to-end approach, the Stable Baselines3 framework was also used. For this model the recurrent version of PPO (Recurrent PPO) [87] was also used, integrating LSTM for better handling of sequential data and CNN for efficient image processing.

The use of Recurrent PPO, incorporating LSTM layers and CNNs, improved the model's ability to navigate the simulation scenario by effectively utilizing past and current images for decision-making.

Table 4: Network Hyperparameters End-to-End Approach

Hyperparameter	Value
Learning Rate	0.0003
Number of Steps	1280
Batch Size	128
Number of Epochs	20
Discount Factor (γ)	0.99
Entropy Coefficient	0.01

The model's architecture is designed to process grayscale images resized to 128x128 pixels, utilizing a convolutional neural network (NatureCNN) for feature extraction. This CNN is composed of several convolutional layers followed by ReLU activation and a flattening operation, leading into an LSTM layer that helps in capturing temporal dependencies. The network concludes with fully connected layers outputting the control actions.

Table 5: Network Architecture End-to-End Approach

Component	Layer	Configuration
Feature Extractor	Conv2d	Conv2d(1, 32, kernel_size=(8, 8), stride=(4, 4)) > ReLU
	Conv2d	Conv2d(32, 64, kernel_size=(4, 4), stride=(2, 2)) > ReLU
	Conv2d	Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1)) > ReLU
Linear Layer	Linear	Linear(in_features=9216, out_features=512) > ReLU
LSTM Layer	LSTM Actor	LSTM(512, 256)
	LSTM Critic	LSTM(512, 256)
Action Network	Linear	Linear(in_features=256, out_features=2)
Value Network	Linear	Linear(in_features=256, out_features=1)

6.6. TRAINING RESULTS

The training was conducted on the same machine taking approximately 34 hours for 147,200 steps. A curriculum learning strategy was also employed, progressively training the model at 50km/h, 70km/h, and finally 90km/h. The rewards per step over these phases are depicted in Figure 31, illustrating the model's learning evolution.

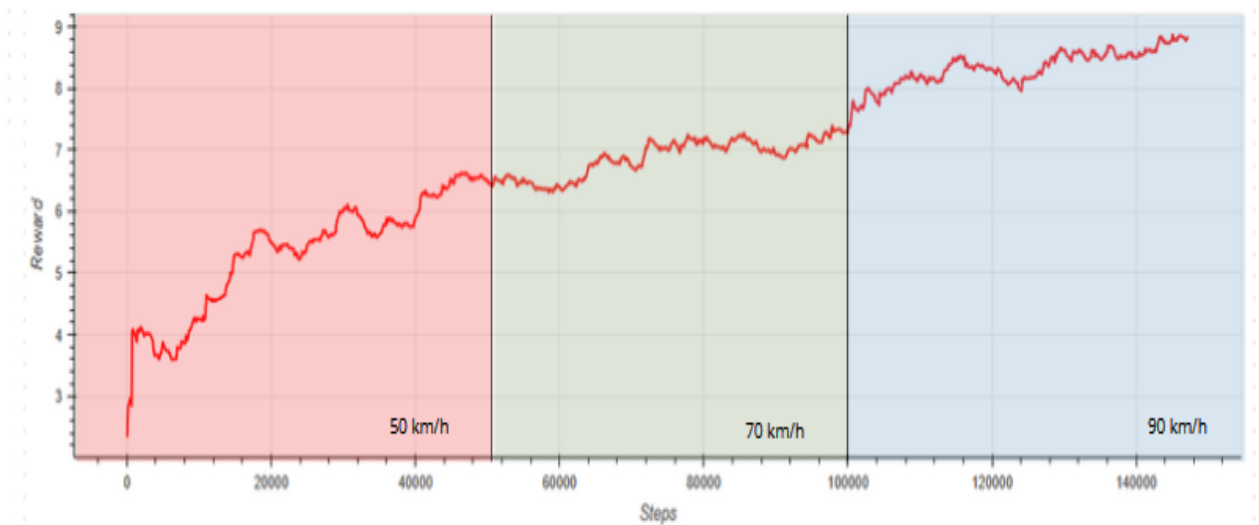


Figure 31: Reward per Step: End-to-End Approach

6.7. TESTING

The learned model was evaluated over 30 episodes for each speed. Figure 32, Figure 33 and Figure 34 showcase the ego car's trajectory at these speeds. At each episode the agent takes a different path, but it effectively avoid the collision on all scenarios. Since the model was motivated to not fully stop, it continues along the path until it reaches the goal position. The testing and the results can be seen on this youtube video⁴.

⁴ [Collision Avoidance - End-to-End Approach - Reinforcement Learning: results](#)

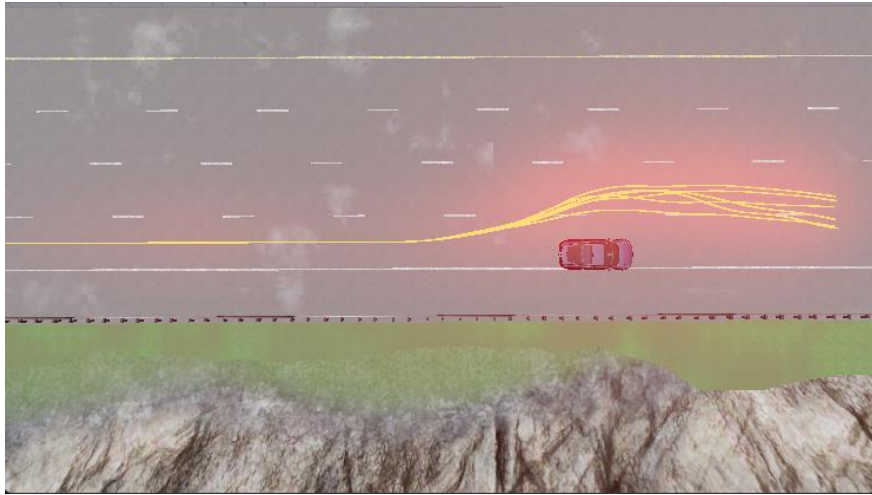


Figure 32: Path followed by ego car at 50 km/h using the end-to-end approach agent.

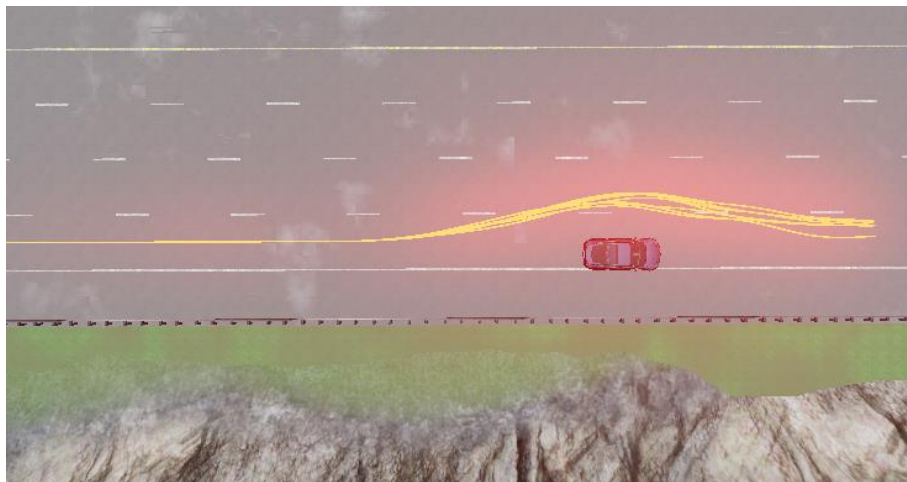


Figure 33: Path followed by ego car at 70 km/h using the end-to-end approach agent.

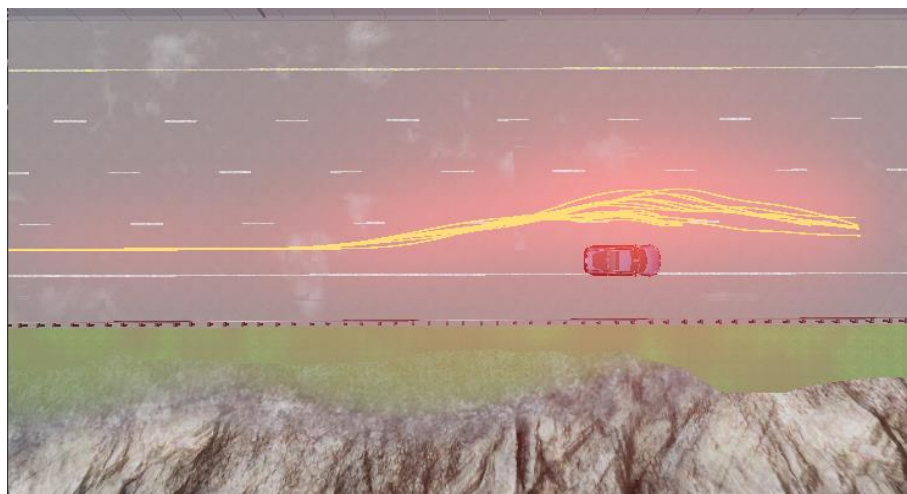


Figure 34: Path followed by ego car at 90 km/h using the end-to-end approach agent.

In the end-to-end model, Figure 35 shows effective collision avoidance through steering.

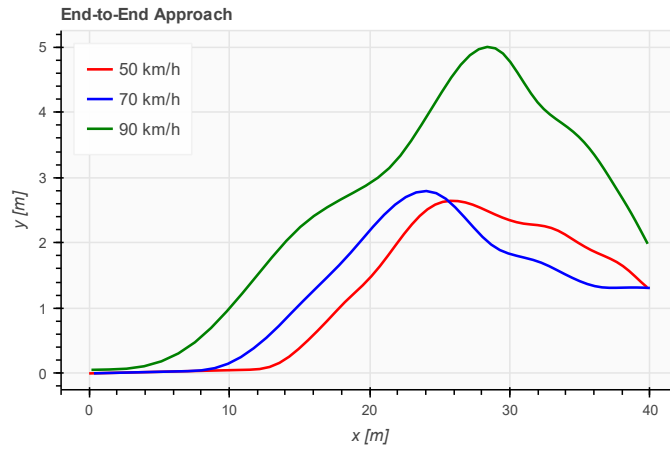


Figure 35: Path taken by the agent at 50, 70 and 90 km/h: End-to-End Approach

According to Figure 36, the car slows to speeds between 20km/h and 35km/h according to its initial speed., here this is also influenced by a reward system favoring distance traveled and penalizing fully barking.

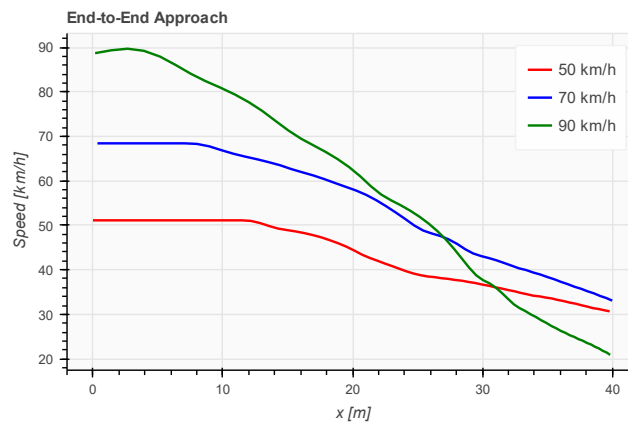


Figure 36: Speed: End-to-End Approach

Figure 37 and Figure 38 reveal that the model's steering and lateral acceleration are irregular, leading to an unstable vehicle trajectory, contrasting with the smoother navigation seen in the MPC-based approach.

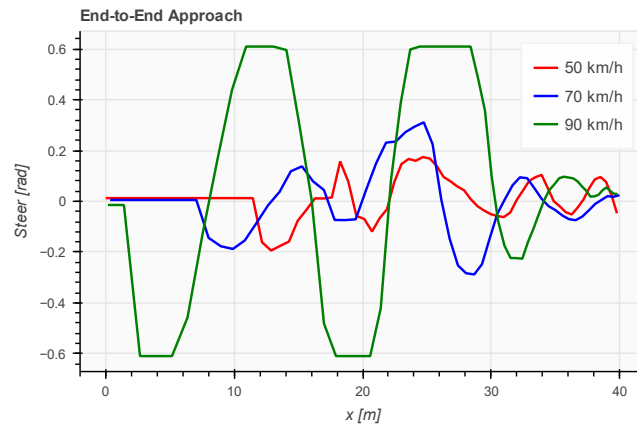


Figure 37: Steer: End-to-End Approach.

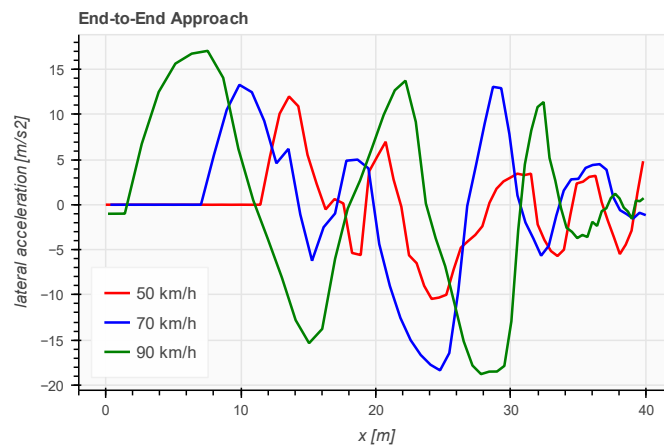


Figure 38: Lateral Acceleration: End-to-End Approach.

7. CONCLUSION AND FUTURE DIRECTIONS

7.1. OBSTACLES AND LIMITATIONS

Both the hybrid and end-to-end (E2E) approaches to collision avoidance systems contains several limitations. A primary concern across both methodologies is the sim-to-real transfer; the models trained in highly controlled simulation environments may not perform as expected in real-world conditions due to unexpected variables. The model is trained on very detailed and specific driving scenarios, which may not cover the vast array of real-world driving conditions.

The state space in the hybrid model is highly descriptive, which while beneficial for accuracy, makes it less adaptable and increases the complexity of real-world application. The E2E model, in comparison, relies heavily on visual inputs and does not account for changes in weather, which can significantly alter visual data and impact model performance.

The success of both approaches centers on the quality and diversity of the training data. A lack of comprehensive and varied data can limit the model's ability to generalize across different driving environments.

Both approaches demonstrate promising results in simulation but require further development and testing to ensure reliability and robustness in diverse, real-world situations. Enhancements in these areas will be crucial for future applications of autonomous driving technologies.

7.2. FUTURE ENHANCEMENTS AND RESEARCH DIRECTIONS

Throughout this project, several potential areas for future research have been identified, to improve the capabilities of the models studied:

- **Sim-to-Real Transition:** Improving techniques to decrease the gap between simulated training environments and real-world conditions is important.
- **Multi-Sensory Integration:** For the E2E approach, incorporating data from additional sensors such as LIDAR or radar could reduce the reliance on visual data alone, particularly in adverse weather conditions.

- **Dynamic and Diverse Scenarios:** Both models would benefit from training across more varied and dynamic scenarios that include different traffic densities, pedestrian interactions, and emergency driving situations.
- **Weather Adaptation for E2E:** Specifically for the E2E approach, training on a more diverse weather condition or integrating weather recognition systems to adapt driving strategies according to real-time weather conditions could improve performance.
- **Advanced Learning Algorithms:** Investigating the potential of advanced reinforcement learning algorithms that can better generalize across different driving styles and more complex decision-making scenarios.
- **Safety Constraints Integration:** Integrating safety constraints within the learning algorithm could ensure safer outcomes, particularly in unforeseen or rare situations.

Addressing these areas not only advances the technology but also increases its reliability, safety, and acceptance in real-world driving applications.

7.3. CONTRIBUTIONS TO THE FIELD

This research contributes to the field of autonomous vehicle technology by demonstrating the feasibility and efficiency of hybrid and end-to-end learning approaches for collision avoidance systems. Key contributions include:

- **Integration of Advanced Learning Techniques:** The implementation of both a hybrid model combining Model Predictive Control (MPC) with Proximal Policy Optimization (PPO) and an end-to-end approach using deep learning addresses complex real-time decision-making challenges in autonomous navigation.
- **Use of Reinforcement Learning:** By adapting and applying the PPO algorithm, and its recurrent variant in practical driving scenarios, this work shows how AI can be effectively utilized to improve vehicular automation.
- **Evaluation in Simulated Environments:** The testing within the CARLA simulator provides insights into the performance and limitations of current AI-driven systems under controlled conditions.

7.4. CLOSING REMARKS

This research has demonstrated the potential and versatility of hybrid and end-to-end approaches in autonomous vehicle technologies. For future, the refinement and testing of these systems in more complex and variable real-world scenarios are important. Learning from the limitations highlighted throughout this project will help to develop more robust, safe, and efficient autonomous driving solutions. The path towards fully autonomous vehicles is complex and has several technical challenges, but the progress we are making offers a promising path forward.

8. REFERENCES

- [1] B. Paden, M. Cap, S. Z. Yong, D. Yershov, and E. Frazzoli, "A Survey of Motion Planning and Control Techniques for Self-driving Urban Vehicles." arXiv, Apr. 25, 2016. doi: 10.48550/arXiv.1604.07446.
- [2] T. Geary and D. Danks, "Balancing the Benefits of Autonomous Vehicles," in *Proceedings of the 2019 AAAI/ACM Conference on AI, Ethics, and Society*, in AIES '19. New York, NY, USA: Association for Computing Machinery, Jan. 2019, pp. 181–186. doi: 10.1145/3306618.3314237.
- [3] Á. Fehér, "Reinforcement learning-based motion planning methods," *Megerősítéses tanulás alapú mozgástervezési módszerek*, 2023, Accessed: Sep. 12, 2023. [Online]. Available: <https://repozitorium.omikk.bme.hu/handle/10890/49885>
- [4] D. Parekh *et al.*, "A Review on Autonomous Vehicles: Progress, Methods and Challenges," *Electronics*, vol. 11, no. 14, Art. no. 14, Jan. 2022, doi: 10.3390/electronics11142162.
- [5] H. Porav and P. Newman, "Imminent Collision Mitigation with Reinforcement Learning and Vision." arXiv, Jan. 03, 2019. doi: 10.48550/arXiv.1901.00898.
- [6] C. van der Ploeg, R. Wouters, M. Baars, A.-L. Do, and F. Fauvel, "Functional Architecture and Implementation of an Autonomous Emergency Steering System." arXiv, Dec. 23, 2022. Accessed: Oct. 30, 2023. [Online]. Available: <http://arxiv.org/abs/2212.12211>
- [7] L. Voesten, "Optimal Trajectory Generation for a Collision Avoidance Maneuver," Eindhoven University of Technology, Eindhoven, 2021.
- [8] Roboauto, "How it works? Collision avoidance system," Medium. Accessed: May 08, 2024. [Online]. Available: <https://blog.roboauto.cz/how-it-works-collision-avoidance-system-d05bd9807f1a>
- [9] S. Rasmana *et al.*, "A Systematic Review on the Autonomous Emergency Steering Assessments and Tests Methodology in ASEAN," *J. Soc. Automot. Eng. Malays.*, vol. 5, pp. 185–193, Sep. 2021, doi: 10.56381/jsaem.v5i2.163.
- [10] C. Ilas, "Electronic sensing technologies for autonomous ground vehicles: A review," presented at the 2013 - 8th International Symposium on Advanced Topics in Electrical Engineering, ATEE 2013, May 2013, pp. 1–6. doi: 10.1109/ATEE.2013.6563528.
- [11] X. Shang and A. Eskandarian, "Emergency Collision Avoidance and Mitigation Using Model Predictive Control and Artificial Potential Function." arXiv, Feb. 08, 2023. Accessed: Sep. 12, 2023. [Online]. Available: <http://arxiv.org/abs/2211.06574>
- [12] A. Seewald, C. Haß, M. Keller, and T. Bertram, "Emergency Steering Assist for Collision Avoidance," *ATZ Worldw.*, vol. 117, no. 1, pp. 14–19, Jan. 2015, doi: 10.1007/s38311-015-0147-1.
- [13] X. He, L. Yulong, C. Lv, X. Ji, and Y. Liu, "Emergency steering control of autonomous vehicle for collision avoidance and stabilisation," *Veh. Syst. Dyn.*, vol. 57, pp. 1163–1187, Nov. 2018, doi: 10.1080/00423114.2018.1537494.
- [14] V. Khattar and A. Eskandarian, "Stochastic Predictive Control for Crash Avoidance in Autonomous Vehicles Based on Stochastic Reachable Set Threat Assessment," presented at the ASME 2021 International Mechanical Engineering Congress and Exposition, American Society of Mechanical Engineers Digital Collection, Jan. 2022. doi: 10.1115/IMECE2021-71179.

- [15] X. Song, H. Gao, and T. Ding, "A Review of the Motion Planning and Control Methods for Automated Vehicles." Accessed: Oct. 14, 2023. [Online]. Available: <https://doi.org/10.3390/s23136140>
- [16] L. Gong, Y. Wu, B. Gao, Y. Sun, X. Le, and C. Liu, "Real-Time Dynamic Planning and Tracking Control of Auto-Docking for Efficient Wireless Charging," *IEEE Trans. Intell. Veh.*, vol. 8, no. 3, pp. 2123–2134, Mar. 2023, doi: 10.1109/TIV.2022.3189511.
- [17] S. Teng *et al.*, "Motion Planning for Autonomous Driving: The State of the Art and Future Perspectives," arXiv.org. Accessed: Oct. 09, 2023. [Online]. Available: <https://arxiv.org/abs/2303.09824v4>
- [18] R. Hoek, J. Ploeg, and H. Nijmeijer, "Cooperative Driving of Automated Vehicles Using B-Splines for Trajectory Planning | IEEE Journals & Magazine | IEEE Xplore." Accessed: Oct. 14, 2023. [Online]. Available: <https://ieeexplore.ieee.org/document/9415170>
- [19] C. V. Samak, T. V. Samak, and S. Kandhasamy, *Control Strategies for Autonomous Vehicles*. 2021. doi: 10.1201/9781003048381.
- [20] B. Oommen, S. Iyengar, N. Rao, and R. Kashyap, "Robot navigation in unknown terrains using learned visibility graphs. Part I: The disjoint convex obstacle case," *IEEE J. Robot. Autom.*, vol. 3, no. 6, pp. 672–681, Dec. 1987, doi: 10.1109/JRA.1987.1087133.
- [21] J. Canny, "A Voronoi method for the piano-movers problem," in *1985 IEEE International Conference on Robotics and Automation Proceedings*, Mar. 1985, pp. 530–535. doi: 10.1109/ROBOT.1985.1087297.
- [22] P. E. Hart, N. J. Nilsson, and B. Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," *IEEE Trans. Syst. Sci. Cybern.*, vol. 4, no. 2, pp. 100–107, Jul. 1968, doi: 10.1109/TSSC.1968.300136.
- [23] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numer. Math.*, vol. 1, no. 1, pp. 269–271, Dec. 1959, doi: 10.1007/BF01386390.
- [24] G. Gordon, "Anytime Dynamic A*: An Anytime, Replanning Algorithm," *Int. Conf. Autom. Plan. Sched. Intell. Plan. Syst.*, Jan. 2005, Accessed: Oct. 14, 2023. [Online]. Available: https://www.academia.edu/24009399/Anytime_Dynamic_A_An_Anytime_Replanning_Algorithm
- [25] D. Ferguson, T. M. Howard, and M. Likhachev, "Motion planning in urban environments," *J. Field Robot.*, vol. 25, no. 11–12, pp. 939–960, Nov. 2008, doi: 10.1002/rob.20265.
- [26] M. Montemerlo *et al.*, "Junior: The Stanford Entry in the Urban Challenge".
- [27] L. De and G. Guglieri, "Advanced Graph Search Algorithms for Path Planning of Flight Vehicles," in *Recent Advances in Aircraft Technology*, R. Agarwal, Ed., InTech, 2012. doi: 10.5772/37033.
- [28] L. Blasi, E. D'Amato, M. Mattei, and I. Notaro, "Path Planning and Real-Time Collision Avoidance Based on the Essential Visibility Graph," *Appl. Sci.*, vol. 10, no. 16, Art. no. 16, Jan. 2020, doi: 10.3390/app10165613.
- [29] T. Krüger, "Graph-Based Speed Planning for Autonomous Driving." Accessed: Oct. 14, 2023. [Online]. Available: https://www.mi.fu-berlin.de/inf/groups/ag-ki/Theses/Completed-theses/Master_Diploma-theses/2019/Krueger/MA_Krueger.pdf
- [30] T. Hegedűs, B. Németh, and P. Gáspár, "Design of a Low-complexity Graph-Based Motion-Planning Algorithm for Autonomous Vehicles," *Appl. Sci.*, vol. 10, no. 21, Art. no. 21, Jan. 2020, doi: 10.3390/app10217716.

- [31] L. Han, H. Yashiro, H. Tehrani Nik Nejad, Q. H. Do, and S. Mita, "Bézier curve based path planning for autonomous vehicle in urban environment," in *2010 IEEE Intelligent Vehicles Symposium*, Jun. 2010, pp. 1036–1042. doi: 10.1109/IVS.2010.5548085.
- [32] V. T. Minh and J. Pumwa, "Feasible Path Planning for Autonomous Vehicles," *Math. Probl. Eng.*, vol. 2014, p. e317494, Mar. 2014, doi: 10.1155/2014/317494.
- [33] X. Li, Z. Sun, Z. He, Q. Zhu, and D. Liu, "A Practical Trajectory Planning Framework for Autonomous Ground Vehicles Driving in Urban Environments," Jun. 2015. doi: 10.1109/IVS.2015.7225840.
- [34] D. Gonzalez Bautista, J. Pérez, V. Milanes, and F. Nashashibi, "A Review of Motion Planning Techniques for Automated Vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 17, pp. 1–11, Nov. 2015, doi: 10.1109/TITS.2015.2498841.
- [35] D. J. Walton and D. S. Meek, "A controlled clothoid spline," *Comput. Graph.*, vol. 29, no. 3, pp. 353–363, Jun. 2005, doi: 10.1016/j.cag.2005.03.008.
- [36] L. Claussmann, M. Revilloud, D. Gruyer, and S. Glaser, "A Review of Motion Planning for Highway Autonomous Driving," *IEEE Trans. Intell. Transp. Syst.*, vol. PP, pp. 1–23, May 2019, doi: 10.1109/TITS.2019.2913998.
- [37] P. Lin, E. Javanmardi, and M. Tsukada, *Clothoid Curve-based Emergency-Stopping Path Planning with Adaptive Potential Field for Autonomous Vehicles*. 2023.
- [38] S. Glaser, B. Vanholme, D. Gruyer, and L. Nouveliere, "Maneuver-Based Trajectory Planning for Highly Autonomous Vehicles on Real Road With Traffic and Driver Interaction," *Intell. Transp. Syst. IEEE Trans. On*, vol. 11, pp. 589–606, Oct. 2010, doi: 10.1109/TITS.2010.2046037.
- [39] C. Yuan *et al.*, "Vehicle Path Planning Based on Pedestrian Collision Avoidance Action," *IEEE Access*, vol. 11, pp. 66713–66728, 2023, doi: 10.1109/ACCESS.2023.3287937.
- [40] L. Zheng, P. Zeng, W. Yang, Y. Li, and Z. Zhan, "Bézier curve-based trajectory planning for autonomous vehicles with collision avoidance," *IET Intell. Transp. Syst.*, vol. 14, no. 13, pp. 1882–1891, 2020, doi: 10.1049/iet-its.2020.0355.
- [41] P. Wang, J. Yang, Y. Zhang, Q. Wang, B. Sun, and D. Guo, "Obstacle-Avoidance Path-Planning Algorithm for Autonomous Vehicles Based on B-Spline Algorithm," *World Electr. Veh. J.*, vol. 13, no. 12, Art. no. 12, Dec. 2022, doi: 10.3390/wevj13120233.
- [42] M. Elbanhawi and M. Simic, "Sampling-Based Robot Motion Planning: A Review," *IEEE Access*, vol. 2, pp. 56–77, 2014, doi: 10.1109/ACCESS.2014.2302442.
- [43] S. Karaman and E. Frazzoli, "Optimal kinodynamic motion planning using incremental sampling-based methods," in *49th IEEE Conference on Decision and Control (CDC)*, Dec. 2010, pp. 7681–7687. doi: 10.1109/CDC.2010.5717430.
- [44] Y. Kuwata, J. Teo, G. Fiore, S. Karaman, E. Frazzoli, and J. How, "Real-Time Motion Planning With Applications to Autonomous Urban Driving," *IEEE*, Aug. 2011.
- [45] S. M. Yang and Y. A. Lin, "Development of an Improved Rapidly Exploring Random Trees Algorithm for Static Obstacle Avoidance in Autonomous Vehicles," *Sensors*, vol. 21, no. 6, Art. no. 6, Jan. 2021, doi: 10.3390/s21062244.
- [46] O. Khatib, "The Potential Field Approach And Operational Space Formulation In Robot Control," in *Adaptive and Learning Systems: Theory and Applications*, K. S. Narendra, Ed., Boston, MA: Springer US, 1986, pp. 367–377. doi: 10.1007/978-1-4757-1895-9_26.

- [47] M. Wolf and J. Burdick, "Artificial potential functions for highway driving with collision avoidance." Accessed: Oct. 15, 2023. [Online]. Available: <https://authors.library.caltech.edu/records/b6f0f-zz238>
- [48] N. Wahid, H. Zamzuri, M. A. Abdul Rahman, S. Kuroda, and P. Raksincharoensak, "Study on potential field based motion planning and control for automated vehicle collision avoidance systems," Feb. 2017, pp. 208–213. doi: 10.1109/ICMECH.2017.7921105.
- [49] Y. Qingfeng *et al.*, "Path Planning Method With Improved Artificial Potential Field—A Reinforcement Learning Perspective," *IEEE Access*, vol. PP, pp. 1–1, Jul. 2020, doi: 10.1109/ACCESS.2020.3011211.
- [50] E. Alcalá, V. Puig, and J. Quevedo, "LPV-MP planning for autonomous racing vehicles considering obstacles," *Robot. Auton. Syst.*, vol. 124, p. 103392, Feb. 2020, doi: 10.1016/j.robot.2019.103392.
- [51] P. Anistratov, *Autonomous Avoidance Maneuvers for Vehicles using Optimization*, vol. 2162. in Linköping Studies in Science and Technology. Dissertations, vol. 2162. Linköping: Linköping University Electronic Press, 2021. doi: 10.3384/diss.diva-176515.
- [52] Z. Chen and X. Huang, "End-to-end learning for lane keeping of self-driving cars," Jun. 2017, pp. 1856–1860. doi: 10.1109/IVS.2017.7995975.
- [53] S. M. H. Jafri and R. Kala, "End-to-end human inspired learning based system for dynamic obstacle avoidance," *Complex Intell. Syst.*, vol. 8, no. 6, pp. 5065–5086, Dec. 2022, doi: 10.1007/s40747-022-00755-0.
- [54] Y. Tian, X. Cao, K. Huang, C. Fei, Z. Zheng, and X. Ji, "Learning to Drive Like Human Beings: A Method Based on Deep Reinforcement Learning," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 7, pp. 6357–6367, Jul. 2022, doi: 10.1109/TITS.2021.3055899.
- [55] L. Qian, X. Xu, Y. Zeng, and J. Huang, "Deep, Consistent Behavioral Decision Making with Planning Features for Autonomous Vehicles," *Electronics*, vol. 8, p. 1492, Dec. 2019, doi: 10.3390/electronics8121492.
- [56] A. O. Ly and M. Akhloufi, "Learning to Drive by Imitation: An Overview of Deep Behavior Cloning Methods," *IEEE Trans. Intell. Veh.*, vol. 6, no. 2, Jun. 2021, Accessed: Oct. 23, 2023. [Online]. Available: <https://trid.trb.org/view/1855127>
- [57] M. Elkholy, K. Nagy, M. Magdy, and H. Ibrahim, "Autonomous Braking and End to End Learning using Single Shot Detection Model and Convolutional Neural Network:," in *Proceedings of the 7th International Conference on Vehicle Technology and Intelligent Transport Systems*, Online Streaming, --- Select a Country ---: SCITEPRESS - Science and Technology Publications, 2021, pp. 309–316. doi: 10.5220/0010398003090316.
- [58] F. Hegedüs, T. Bécsi, S. Aradi, and P. Gáspár, "Motion Planning for Highly Automated Road Vehicles with a Hybrid Approach Using Nonlinear Optimization and Artificial Neural Networks," *Számítástechnikai és Automatizálási Tudományok Magyar Társasága Szimpóziuma*, vol. 65, pp. 148–160, Mar. 2019, doi: 10.5545/sv-jme.2018.5802.
- [59] M. P. Ronecker and Y. Zhu, "Deep Q-Network Based Decision Making for Autonomous Driving," in *2019 3rd International Conference on Robotics and Automation Sciences (ICRAS)*, Jun. 2019, pp. 154–160. doi: 10.1109/ICRAS.2019.8808950.
- [60] S. Aradi, "Survey of Deep Reinforcement Learning for Motion Planning of Autonomous Vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, pp. 740–759, Feb. 2022, doi: 10.1109/TITS.2020.3024655.

- [61] M. M. S. Ibrahim, M. R. Atia, and M. W. Fakhr, "Autonomous Vehicle Path Planning using Q-Learning," *J. Phys. Conf. Ser.*, vol. 2128, no. 1, p. 012018, Dec. 2021, doi: 10.1088/1742-6596/2128/1/012018.
- [62] Y. Qian, S. Feng, W. Hu, and W. Wang, "Obstacle avoidance planning of autonomous vehicles using deep reinforcement learning," *Adv. Mech. Eng.*, vol. 14, no. 12, p. 16878132221139661, Dec. 2022, doi: 10.1177/16878132221139661.
- [63] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "CARLA: An Open Urban Driving Simulator." arXiv, Nov. 10, 2017. doi: 10.48550/arXiv.1711.03938.
- [64] Á. Fehér, S. Aradi, and T. Bécsi, "Online Trajectory Planning with Reinforcement Learning for Pedestrian Avoidance," *Electronics*, vol. 11, no. 15, Art. no. 15, Jan. 2022, doi: 10.3390/electronics11152346.
- [65] Á. Fehér, S. Aradi, T. Bécsi, P. Gáspár, and Z. Szalay, "Proving Ground Test of a DDPG-based Vehicle Trajectory Planner," May 2020, pp. 332–337. doi: 10.23919/ECC51009.2020.9143675.
- [66] M. Schratter, M. Hartmann, and D. Watzenig, "Pedestrian Collision Avoidance System for Autonomous Vehicles," *SAE Int. J. Connect. Autom. Veh.*, vol. 2, no. 4, Art. no. 12-02-04–0021, Dec. 2019, doi: 10.4271/12-02-04-0021.
- [67] X. Zong, G. Xu, G. Yu, H. Su, and C. Hu, "Obstacle Avoidance for Self-Driving Vehicle with Reinforcement Learning," *SAE Int. J. Passeng. Cars - Electron. Electr. Syst.*, vol. 11, no. 1, pp. 30–39, Sep. 2017, doi: 10.4271/07-11-01-0003.
- [68] N. J. Zakaria, M. I. Shapiai, and N. Wahid, "A study of multiple reward function performances for vehicle collision avoidance systems applying the DQN algorithm in reinforcement learning," *IOP Conf. Ser. Mater. Sci. Eng.*, vol. 1176, no. 1, p. 012033, Aug. 2021, doi: 10.1088/1757-899X/1176/1/012033.
- [69] "Euro NCAP 2025 Roadmap." Accessed: Oct. 29, 2023. [Online]. Available: <https://cdn.euroncap.com/media/30700/euroncap-roadmap-2025-v4.pdf>
- [70] "WP.29 - Introduction | UNECE." Accessed: Nov. 04, 2023. [Online]. Available: <https://unece.org/wp29-introduction>
- [71] "UN Vehicle Regulations." Accessed: Nov. 04, 2023. [Online]. Available: <https://unece.org/sites/default/files/2022-01/UNECE%20Vehicle%20Regulations.pdf>
- [72] "UN regulation 152:Advanced Emergency Braking System (AEBS)."
- [73] "UN regulation 79: Steering Equipment."
- [74] A. Rigling and J. Ellway, "Test of Emergency Steering Support Systems Euro NCAP." Jan. 01, 2023. Accessed: Oct. 30, 2023. [Online]. Available: <https://cdn.euroncap.com/media/68587/tb-037-ess-assessment-v10.pdf>
- [75] "ASSESSMENT PROTOCOL – SAFETY ASSIST – COLLISION AVOIDANCE." Feb. 2022. Accessed: Nov. 01, 2023. [Online]. Available: <https://cdn.euroncap.com/media/67897/euro-ncap-assessment-protocol-sa-collision-avoidance-v100.pdf>
- [76] "TEST PROTOCOL – AEB VRU systems." Accessed: Nov. 01, 2023. [Online]. Available: <https://cdn.euroncap.com/media/26997/euro-ncap-aeb-vru-test-protocol-v20.pdf>
- [77] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms." arXiv, Aug. 28, 2017. doi: 10.48550/arXiv.1707.06347.
- [78] "Welcome to CVXPY 1.4 — CVXPY 1.4 documentation." Accessed: Feb. 04, 2024. [Online]. Available: <https://www.cvxpy.org/>
- [79] "Vehicles — NVIDIA PhysX SDK 3.4.0 Documentation." Accessed: Mar. 02, 2024. [Online]. Available:

- <https://docs.nvidia.com/gameworks/content/gameworkslibrary/physx/guide/Manual/Vehicles.html>
- [80] M.-A. Chadi and H. Mousannif, “Understanding Reinforcement Learning Algorithms: The Progress from Basic Q-learning to Proximal Policy Optimization”.
 - [81] S. Dey, T. Marzullo, X. Zhang, and G. Henze, “Reinforcement learning building control approach harnessing imitation learning,” *Energy AI*, vol. 14, p. 100255, Oct. 2023, doi: 10.1016/j.egyai.2023.100255.
 - [82] B. Peng *et al.*, “End-to-End Autonomous Driving Through Dueling Double Deep Q-Network,” *Automot. Innov.*, vol. 4, no. 3, pp. 328–337, Aug. 2021, doi: 10.1007/s42154-021-00151-3.
 - [83] “Cubic spline planning — PythonRobotics documentation.” Accessed: Feb. 18, 2024. [Online]. Available: https://atsushisakai.github.io/PythonRobotics/modules/path_planning/cubic_spline/cubic_spline.html
 - [84] Y. Zhang and Z. Huo, “Carla-MPC.” Accessed: Feb. 22, 2024. [Online]. Available: <https://github.com/yangggzhang/Carla-MPC>
 - [85] J. Kong, M. Pfeiffer, G. Schildbach, and F. Borrelli, *Kinematic and dynamic vehicle models for autonomous driving control design*. 2015, p. 1099. doi: 10.1109/IVS.2015.7225830.
 - [86] “Stable-Baselines3 Docs - Reliable Reinforcement Learning Implementations — Stable Baselines3 2.3.0a3 documentation.” Accessed: Mar. 02, 2024. [Online]. Available: <https://stable-baselines3.readthedocs.io/en/master/index.html>
 - [87] “Recurrent PPO — Stable Baselines3 - Contrib 2.3.0a1 documentation.” Accessed: Mar. 02, 2024. [Online]. Available: https://sb3-contrib.readthedocs.io/en/master/modules/ppo_recurrent.html

9. LIST OF FIGURES

Figure 1: Use cases in which an AES system can mitigate dangerous scenarios where an AEB system may fail [6].....	8
Figure 2: Example of FCW and AEB [8].....	10
Figure 3: Algorithm flow of collision avoidance system by applying AES [9].....	11
Figure 4: High-level diagram of the control system of a collision avoidance system [10].	11
Figure 5: Pipeline and end-to-end frameworks [17].	13
Figure 6: Bézier curve interpolation for obstacle avoidance [31].....	16
Figure 7: Potential Field in a Highway [11].....	19
Figure 8: ESS Test diagram using CCRs -50 scenario [74].....	33
Figure 9: ESS Test diagram using CPLA25 scenario [74].	33
Figure 10: Simulation Scenario.....	41
Figure 11: The cosine of the difference between the yaw of the ego vehicle and the yaw corresponding to the correct orientation on the road.	44
Figure 12: The distance of the ego vehicle from the optimal path or the lane center.	45
Figure 13: Collision Indicator	45
Figure 14: Lane Invasion Indicator	45
Figure 15: The distance traveled.	46
Figure 16: Proposed Hybrid Collision Avoidance Framework.....	47
Figure 17: State Space.....	50
Figure 18: Action Space.....	51
Figure 19: Cubic Spline Curve.....	53
Figure 20: Kinematic Bicycle Model [85]	56
Figure 21: Reward per Step: Hybrid Approach	60
Figure 22: Path followed by ego car at 50 km/h using the hybrid approach agent.	61
Figure 23: Path followed by ego car at 70 km/h using the hybrid approach agent.	61
Figure 24: Path followed by ego car at 90 km/h using the hybrid approach agent.	62
Figure 25: Path taken by the agent at 50, 70 and 90 km/h: Hybrid Approach.....	62
Figure 26: Speed: Hybrid Approach	63
Figure 27: Steer: Hybrid Approach.	63
Figure 28: Lateral Acceleration: Hybrid Approach.	63
Figure 29: Proposed End-to-End Collision Avoidance Framework	64
Figure 30: RGB camera pre-processing.	66
Figure 31: Reward per Step: End-to-End Approach	68
Figure 32: Path followed by ego car at 50 km/h using the end-to-end approach agent. ...	69
Figure 33: Path followed by ego car at 70 km/h using the end-to-end approach agent. ...	69
Figure 34: Path followed by ego car at 90 km/h using the end-to-end approach agent. ..	69
Figure 35: Path taken by the agent at 50, 70 and 90 km/h: End-to-End Approach.....	70
Figure 36: Speed: End-to-End Approach.....	70
Figure 37: Steer: End-to-End Approach.	71
Figure 38: Lateral Acceleration: End-to-End Approach.	71

10. LIST OF TABLES

Table 1: Advantages and Disadvantages of each mentioned approach.	27
Table 2: Network Hyperparameters Hybrid Approach	59
Table 3: Network Architecture.....	60
Table 4: Network Hyperparameters End-to-End Approach.....	67
Table 5: Network Architecture End-to-End Approach	67

11. APPENDIX

11.1. LINEAR MPC IMPLEMENTATION

```
def linear_mpc(self, z_ref, z_initial, prev_deltas, dt):
    z = cvxpy.Variable((4, self.len_horizon + 1))
    u = cvxpy.Variable((2, self.len_horizon))

    cost = 0
    constraints = [z[:, 0] == z_initial.flatten()]
    for i in range(self.len_horizon - 1):
        ## Cost
        if i != 0:
            cost += cvxpy.quad_form(z_ref[:, i] - z[:, i], self.Q)
            cost += cvxpy.quad_form(u[:, i] - u[:, i-1], self.Rd)
        else:
            u_prev = [self.prev_accelerations[0], self.prev_deltas[0]]
            cost += cvxpy.quad_form(u[:, i] - u_prev, self.Rd)

        cost += cvxpy.quad_form(u[:, i], self.R)

        ## Constraints
        A, B, C = self.get_linearized_dynamics(z_ref[3, i],
self.prev_deltas[np.min([ i + 1, len(self.prev_deltas) - 1])],
z_ref[2, i], dt)

        constraints += [z[:, i+1] == A @ z[:, i] + B @ u[:, i] +
C.flatten()]

    # Velocity limits
    constraints += [z[2, i] <= self.v_max]
    constraints += [z[2, i] >= self.v_min]

    # Input limits
    constraints += [self.a_min <= u[0, i]]
    constraints += [u[0, i] <= self.a_max]
    constraints += [u[1, i] <= self.max_steering_angle]
    constraints += [u[1, i] >= -self.max_steering_angle]

    # Rate of change of input limit
    if i != 0:
        constraints += [u[0, i] - u[0, i-1] <= self.a_rate_max]
        constraints += [u[0, i] - u[0, i-1] >= -self.a_rate_max]
        constraints += [u[1, i] - u[1, i-1] <= self.steer_rate_max
* dt]
```

```

        constraints += [u[1, i] - u[1, i-1] >= -
self.steer_rate_max * dt]

    # Terminal cost
    cost += cvxpy.quad_form(z_ref[:, -1] - \
                            z[:, -1], self.Qf)

    # Quadratic Program
    qp = cvxpy.Problem(cvxpy.Minimize(cost), constraints)
    qp.solve(solver=cvxpy.ECOS, verbose=False)

    # print(qp.status)
    self.qp_status = str(qp.status)
    # print(self.qp_status)

    if qp.status == cvxpy.OPTIMAL or qp.status ==
cvxpy.OPTIMAL_INACCURATE:
        x = np.array(z.value[0, :]).flatten()
        y = np.array(z.value[1, :]).flatten()
        v = np.array(z.value[2, :]).flatten()
        yaw = np.array(z.value[3, :]).flatten()
        a = np.array(u.value[0, :]).flatten()
        delta = np.array(u.value[1, :]).flatten()
    else:
        # x, y, v, yaw, a, delta = None, None, None, None, None, None
        a, delta = None, None

    return a, delta, self.qp_status

```

11.2. CODE FOR LINEARIZATION

```

syms v psi lr lf delta a L x y
f = [ v*cos(psi + atan(lr/L*tan(delta))); v*sin(psi + atan(lr/L*tan(delta)));
a; v/lr*sin(atan(lr/L*tan(delta)))]];
A = jacobian(f, [x; y; v; psi]);
B = jacobian(f, [a; delta]);
X = [x; y; v; psi];
U = [a; delta];
C = f - A*X - B*U;

```